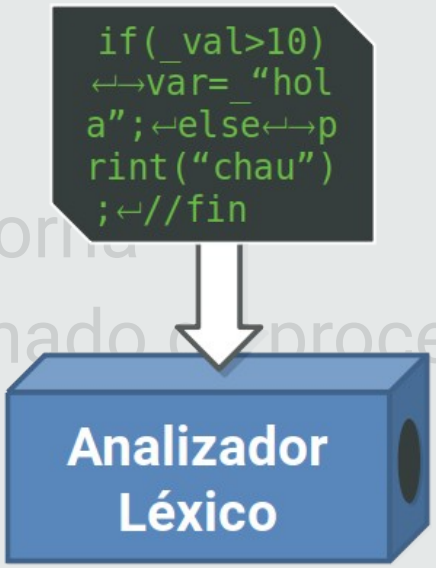
Compiladores e Intérpretes

Analizador Léxico – AF y Consideraciones de Diseño

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

Consideraciones Generales

- Para implementar el **Analizador Léxico** vamos a usar los AF que reconocen las ER asociadas a cada token del lenguaje
- Tenemos que tener en cuenta sus **funcionalidades generales**:
  - Recibe y va leyendo el programa fuente
  - Funciona por demanda
  - Ante un pedido, reconoce un token y lo retorna
  - Al retornar un token puede no haber terminado de procesar el programa fuente



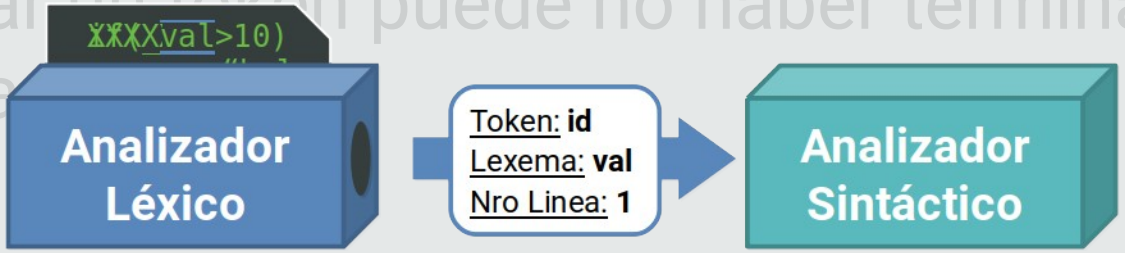
Consideraciones Generales

- Para implementar el **Analizador Léxico** vamos a usar los AF que reconocen las ER asociadas a cada token del lenguaje
- Tenemos que tener en cuenta sus **funcionalidades generales**:
  - Recibe y va leyendo el programa fuente
  - Funciona por demanda
  - Ante un p noce un token y lo retorna
  - Al retor ede no haber te cesar el program



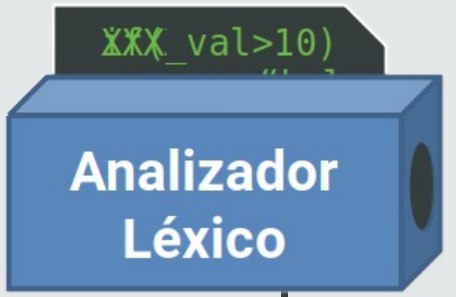
Consideraciones Generales

- Para implementar el **Analizador Léxico** vamos a usar los AF que reconocen las ER asociadas a cada token del lenguaje
- Tenemos que tener en cuenta sus **funcionalidades generales**:
  - Recibe y va leyendo el programa fuente
  - Funciona por demanda
  - Ante un pedido, reconoce un token y lo retorna
  - Al retornar un token puede no haber terminado de procesar el programa



Consideraciones Generales

- Para implementar el **Analizador Léxico** vamos a usar los AF que reconocen las ER asociadas a cada token del lenguaje
- Tenemos que tener en cuenta sus **funcionalidades generales**:
  - Recibe y va leyendo el programa fuente
  - Funciona por demanda
  - Ante un pedido, reconoce un token y lo retorna
  - Al retornar un token puede no haber terminado de procesar el programa fuente



Reconocer cualquier Token

- Para **implementar** el **analizador léxico** de un lenguaje de programación tenemos que **identificar** sus **tokens**
- Para esto, para cada token, tenemos que determinar el **patrón** que caracteriza a sus **lexemas**

Los especificamos mediante **Expresiones Regulares**

Reconocer cualquier Token

- Para **reconocer** estos patrones en una cadena (archivo de entrada) nos valemos de las **propiedades** de los **lenguajes regulares**, los cuales:

Son reconocidos mediante **Autómatas Finitos**

Reconocer cualquier Token

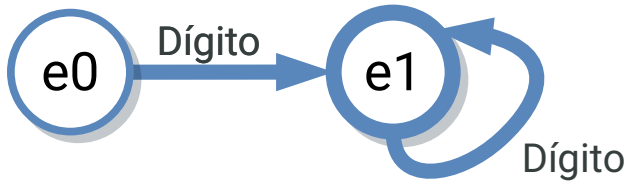
- Ante un **pedido** el Analizador Léxico debería poder reconocer **cualquier token** válido

el **Analizador Léxico** se basará en un **AF** que se construye **uniendo** adecuadamente los AFs de cada token

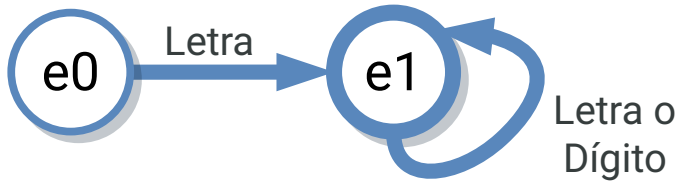
Reconocer cualquier Token

- Ante un **pedido** el Analizador Léxico debería poder reconocer **cualquier token** válido

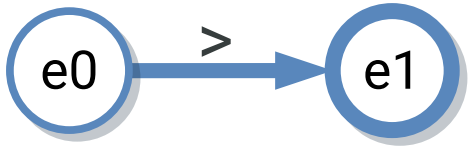
Enteros



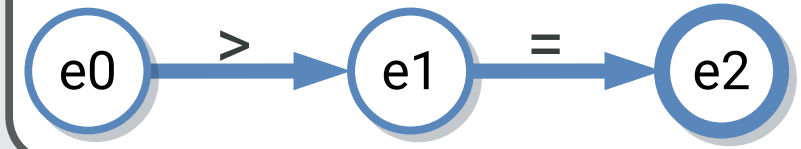
Identificadores



Operador Mayor



Operador Mayor o Igual

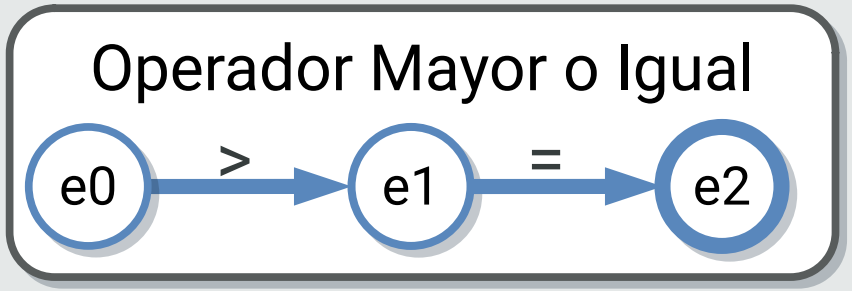


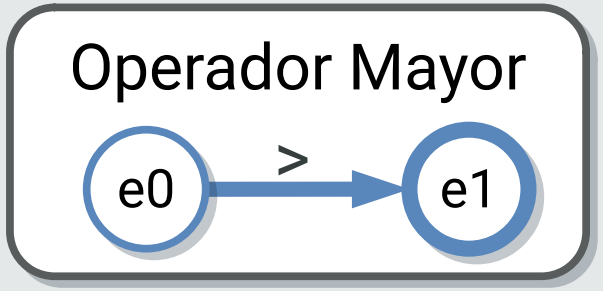
Supongamos un lenguaje pequeño donde los tokens solo pueden ser enteros, identificadores, mayor y mayor o igual

¿Cómo seria el AF para su Analizador Léxico?

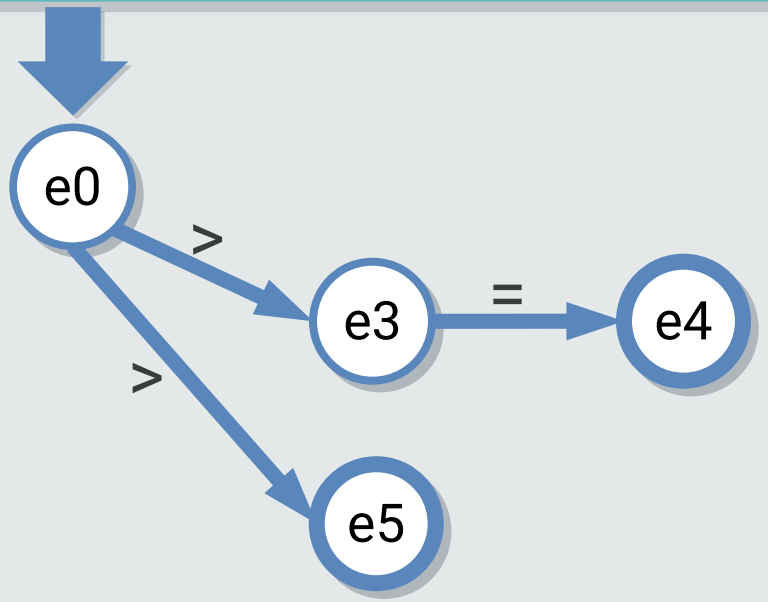
No Determinismo

- Hay tokens que comparten prefijos



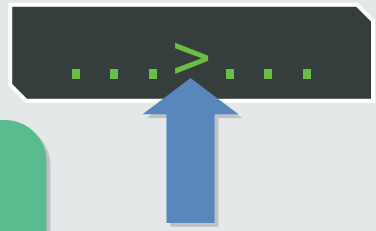






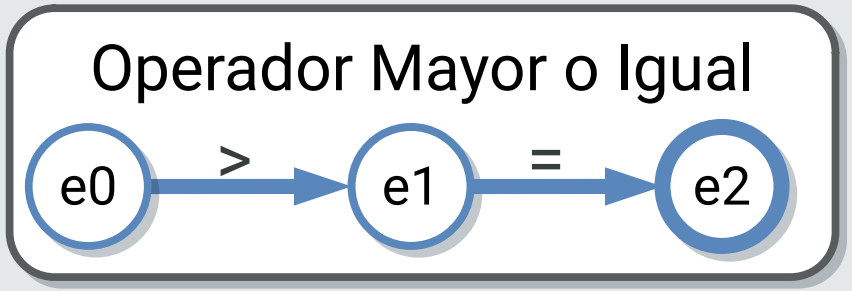
Como vimos recién esos dos AF se fusionaron de esa manera

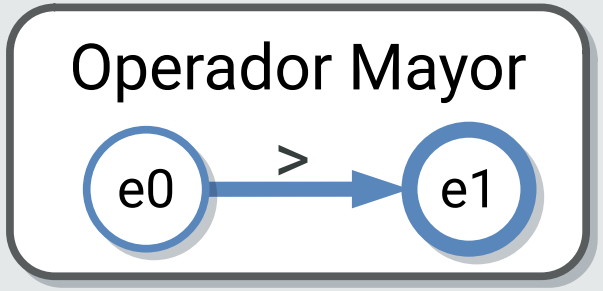
¿Qué pasa si el próximo carácter a procesar es un >?

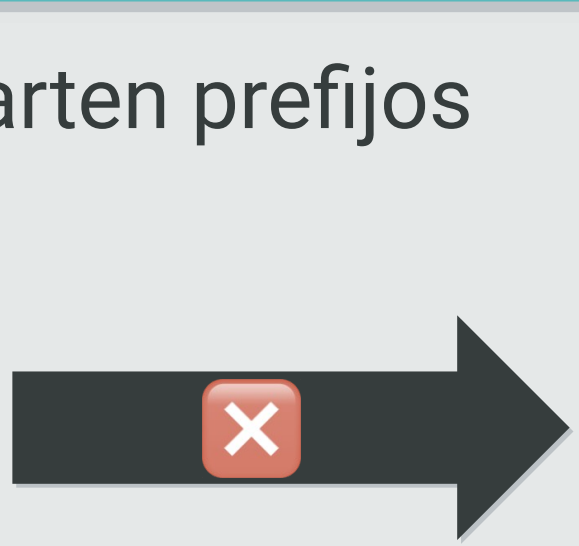


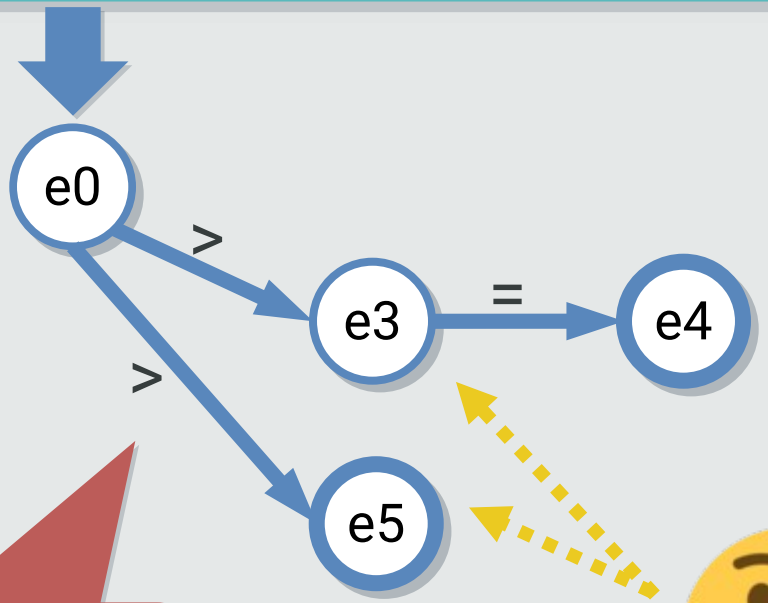
No Determinismo

- Hay tokens que comparten prefijos



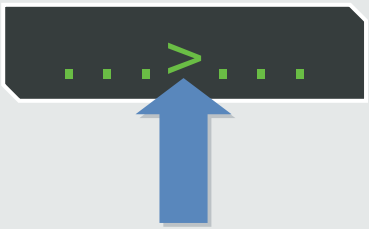






No determinista!

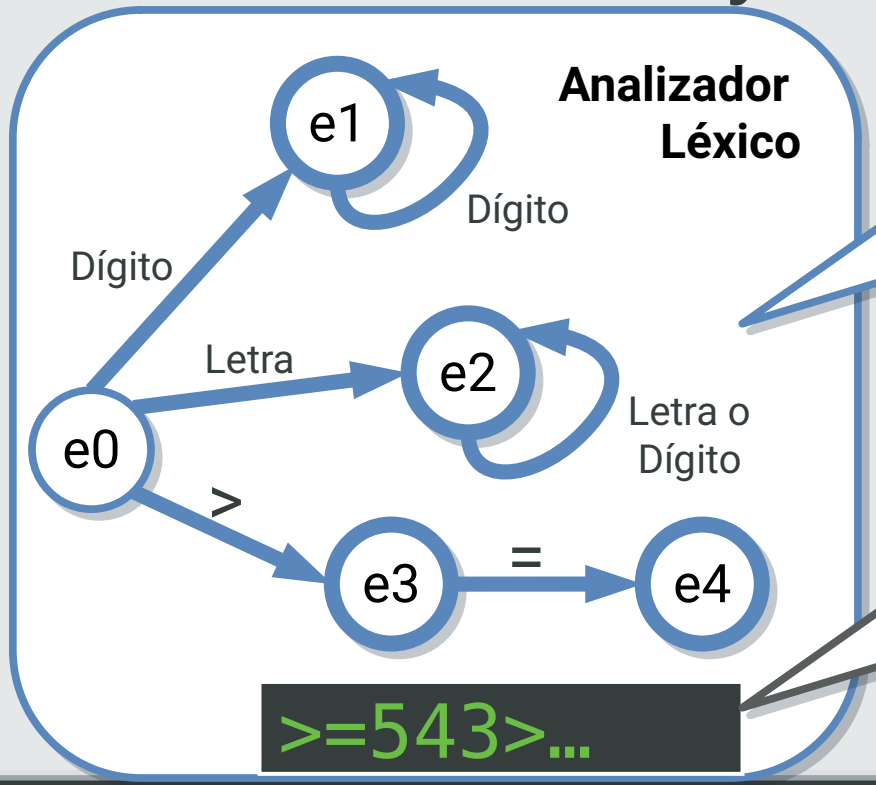




No sabemos si ir a e3 o a e5!

Pedidos y Retornos de Tokens

- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**

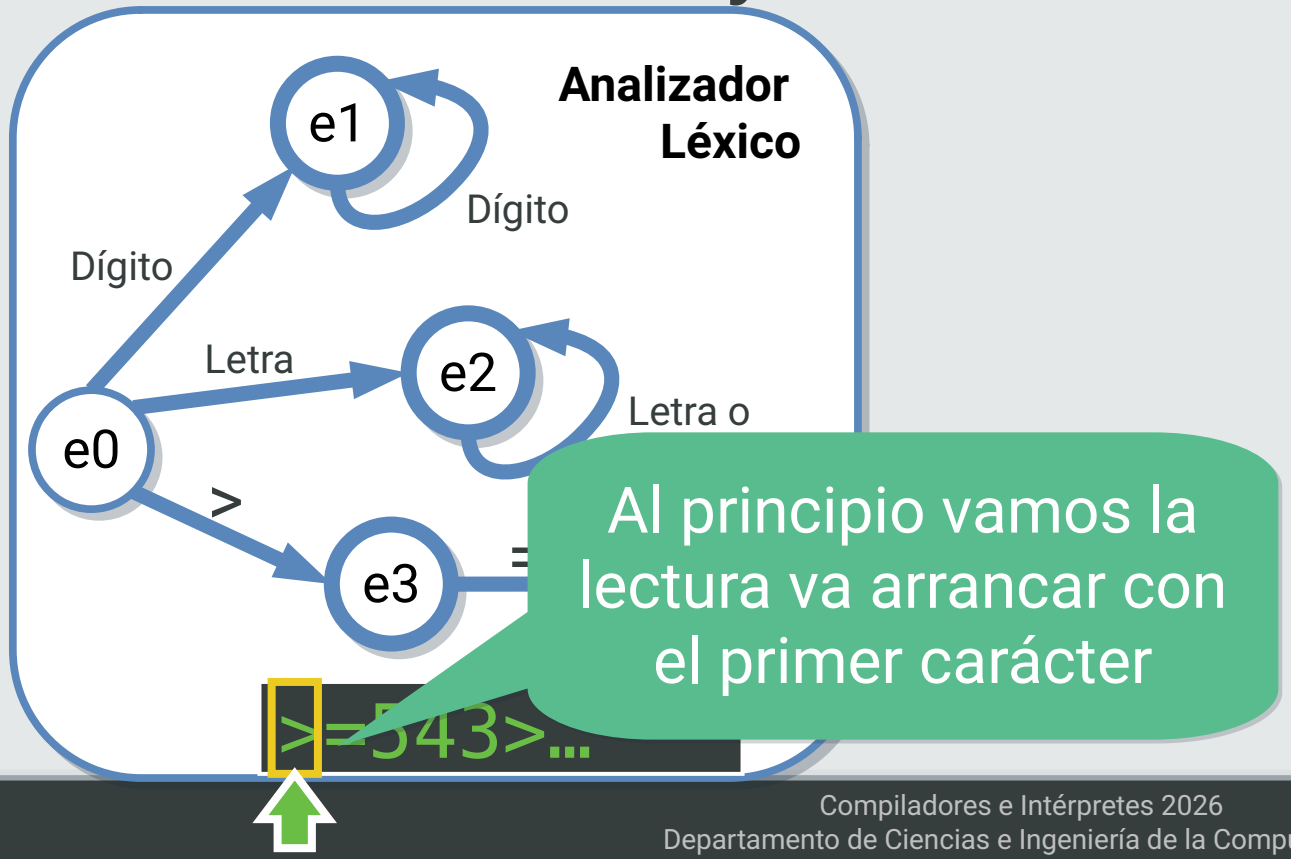


Supongamos nuestro analizador léxico esta implementado de alguna manera usando el AF que veníamos armando

Supongamos que este es programa fuente

Pedidos y Retornos de Tokens

- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



Pedidos y Retornos de Tokens

Cuando tenemos un pedido de token, como dijimos antes arrancamos de el e0 y empezamos va procesar caracteres del fuente

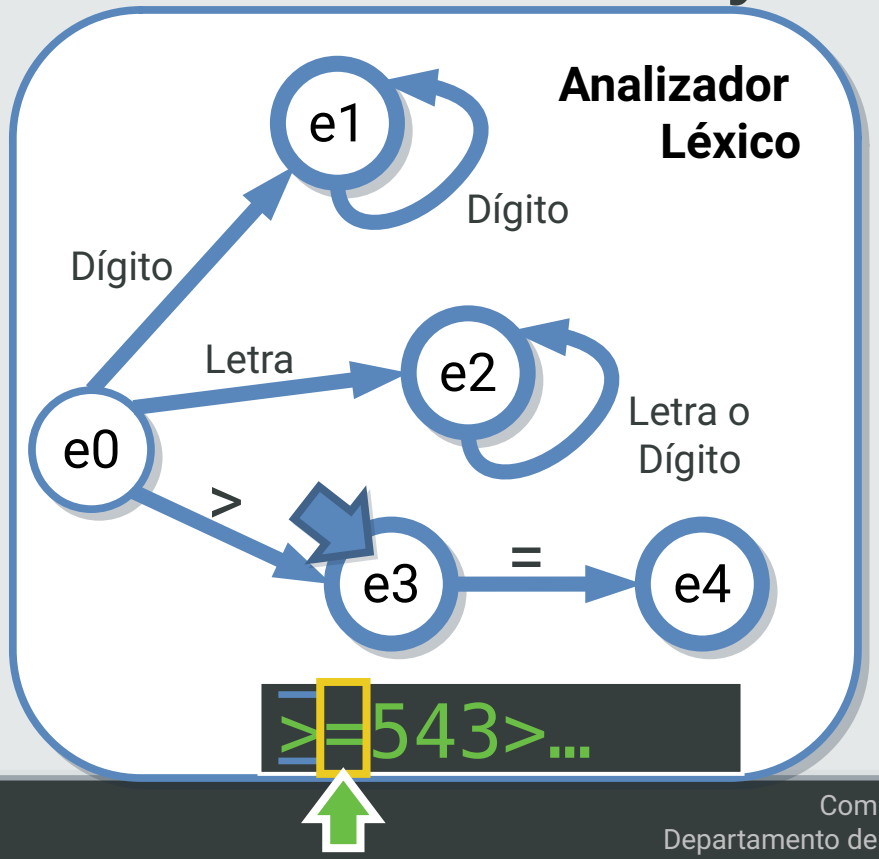
contener **mas de un token**, pero
etorna de a uno a la vez



Next Token!

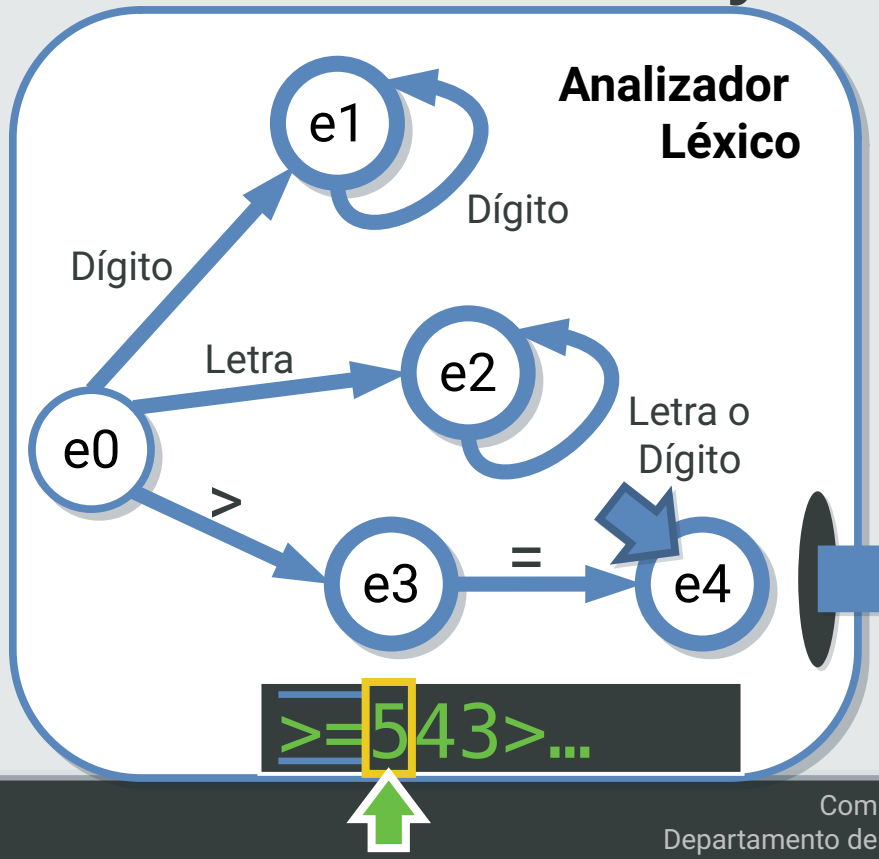
Pedidos y Retornos de Tokens

- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



Pedidos y Retornos de Tokens

- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



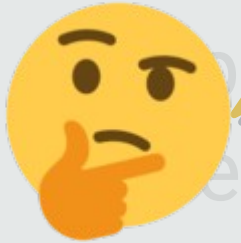
Token: **MayorIgual**
Lexema: **>=**
Nro Linea: **1**

Cuando llego al estado terminador y termine de armar el token debo retornarlo!

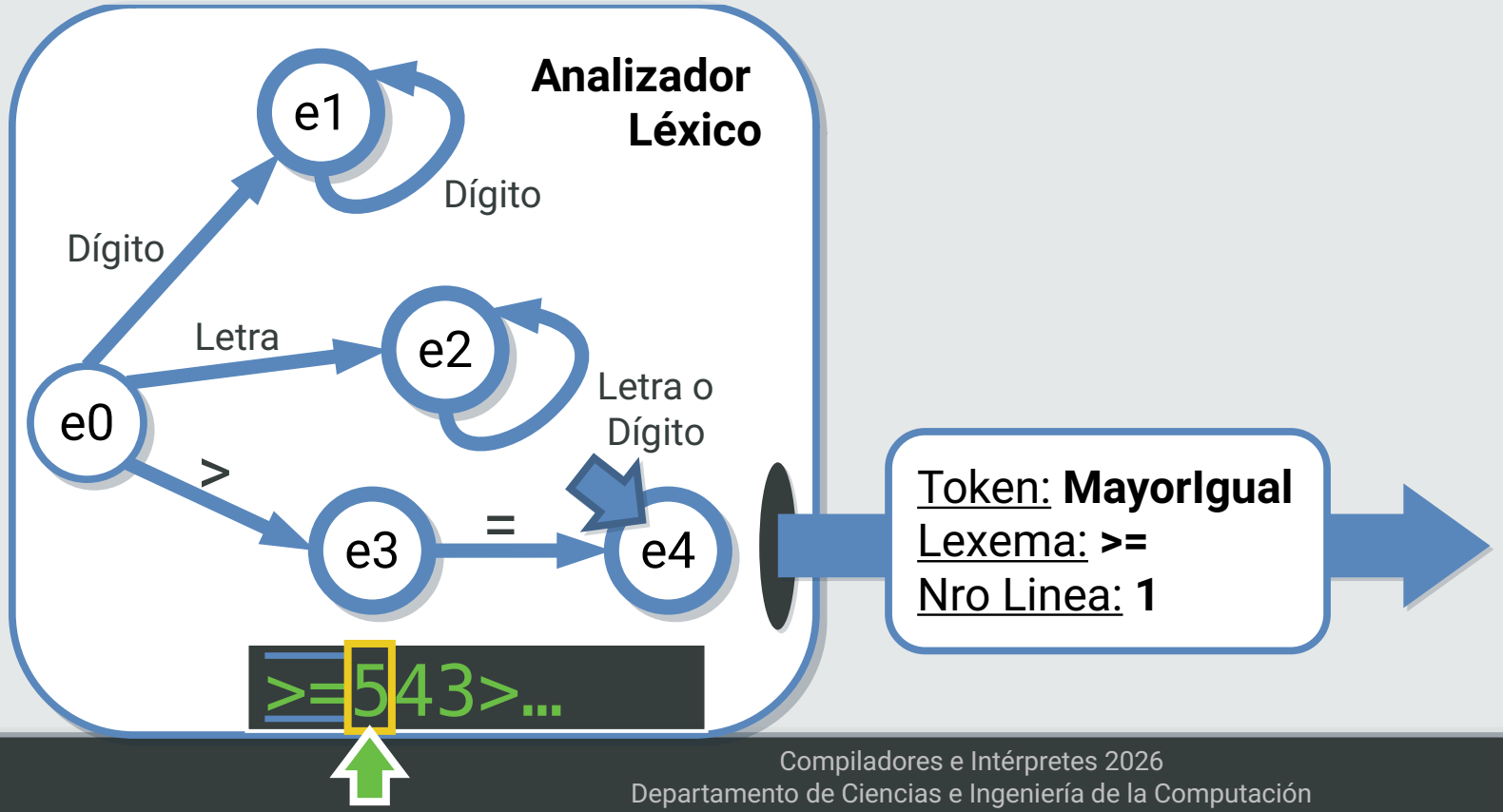
Incluyendo su **lexema** y nro de Linea

de Tokens

Bien… Pero, antes también pase por el estado terminador e3 y no retornar el token “>”… ¿Cómo se cuando hay de retornarlo?



…as de un token, pero
…estro AF reconoce y retorna de a uno a la vez



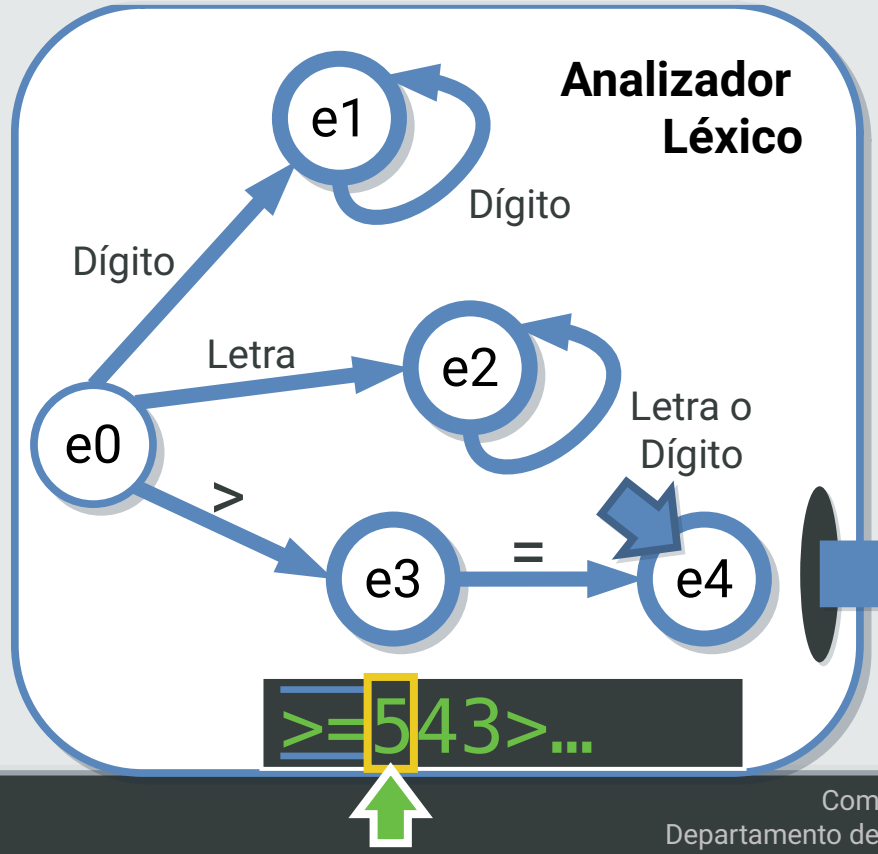
Token: **MayorIgual**
Lexema: **>=**
Nro Linea: **1**

de Tokens

Bien… Pero, antes también pase por el estado terminador e3 y no retornar el token “>”… ¿Cómo se cuando hay de retornarlo?

La intuición es **armar el token mas grande** posible!

cuando **no podes agregarle mas** caracteres y estas en un estado terminador ahí lo **retornas**!

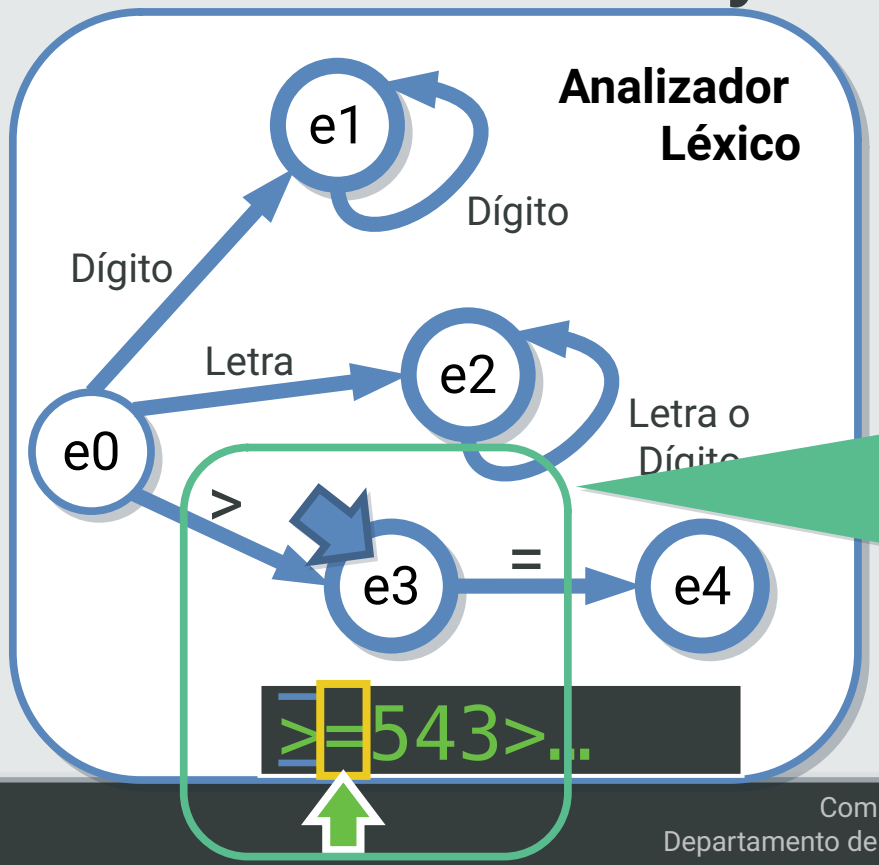


Token: **MayorIgual**
Lexema: **>=**
Nro Linea: **1**

Volvamos un paso atrás para ver esto en detalle

Pedidos y Retornos de Tokens

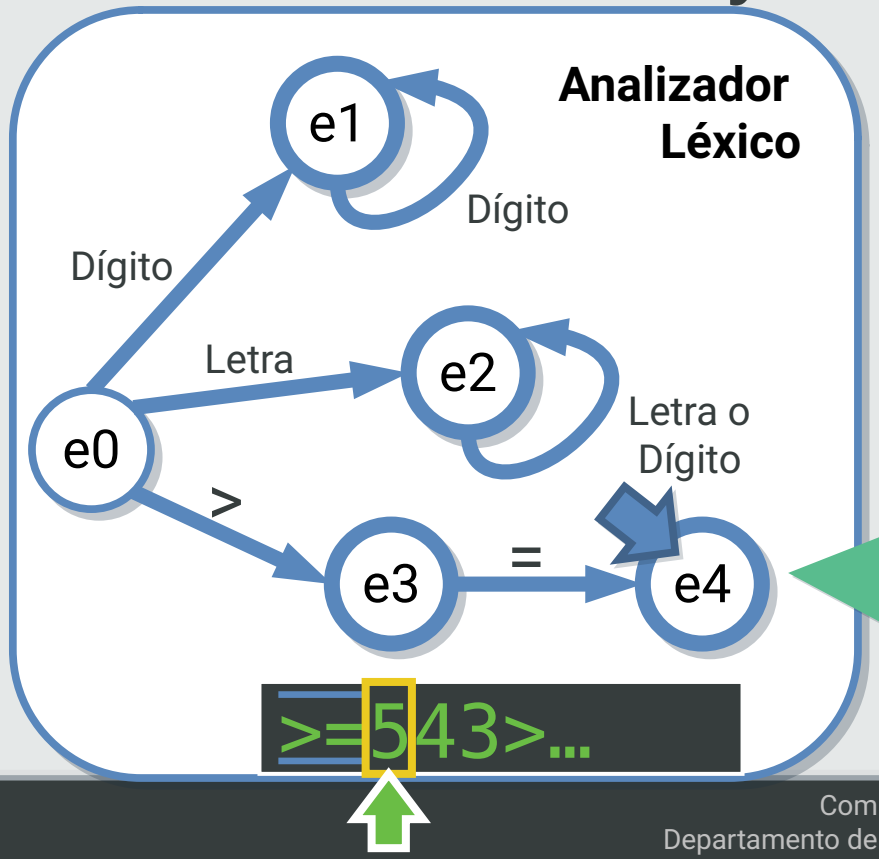
- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



En esta situación si bien estamos en un estado terminador, puedo seguir agregando caracteres al lexema del token ya que estamos viendo el "=" y desde e3 puedo ir a e4 con ese carácter

Pedidos y Retornos de Tokens

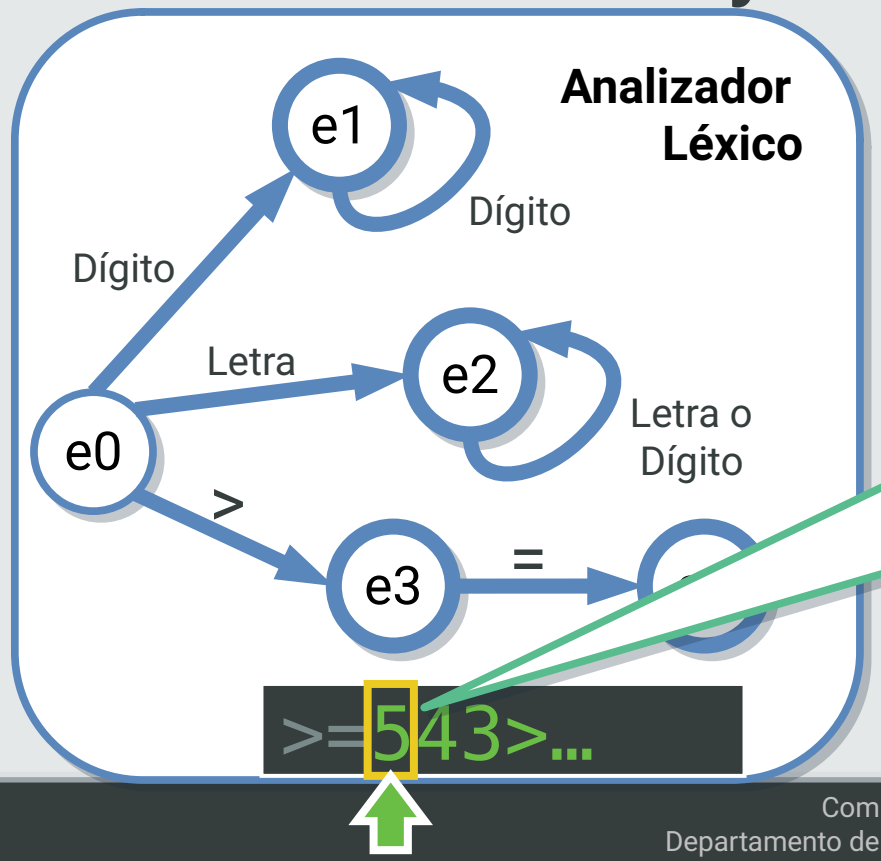
- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



En este caso si tengo que retornar, porque e4 es un estado terminador y no puedo ir a ningún lado desde e4 con el carácter “5” (que estoy viendo ahora)

Pedidos y Retornos de Tokens

- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



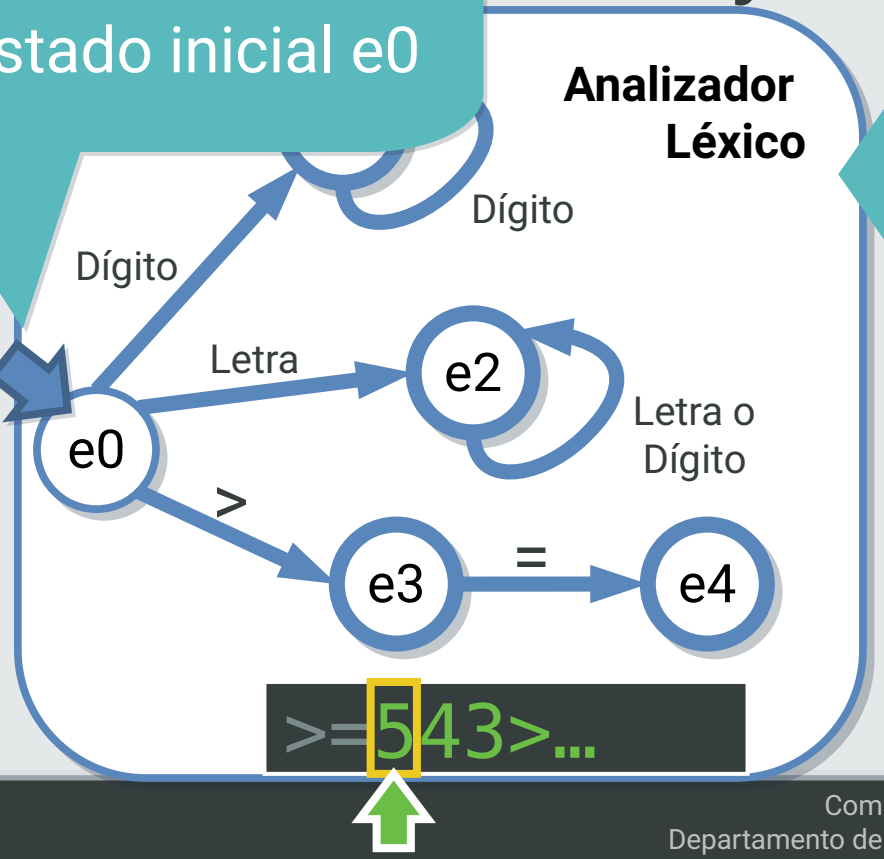
Una vez que retorno el token, el analizador mantiene el estado del lugar donde estaba "viendo" en el programa fuente

Esto es importante porque en el próximo pedido tiene que arrancar a leer de ahí!

Pedidos y Retornos de Tokens

Nuevamente, ante un pedido arrancamos del estado inicial e0

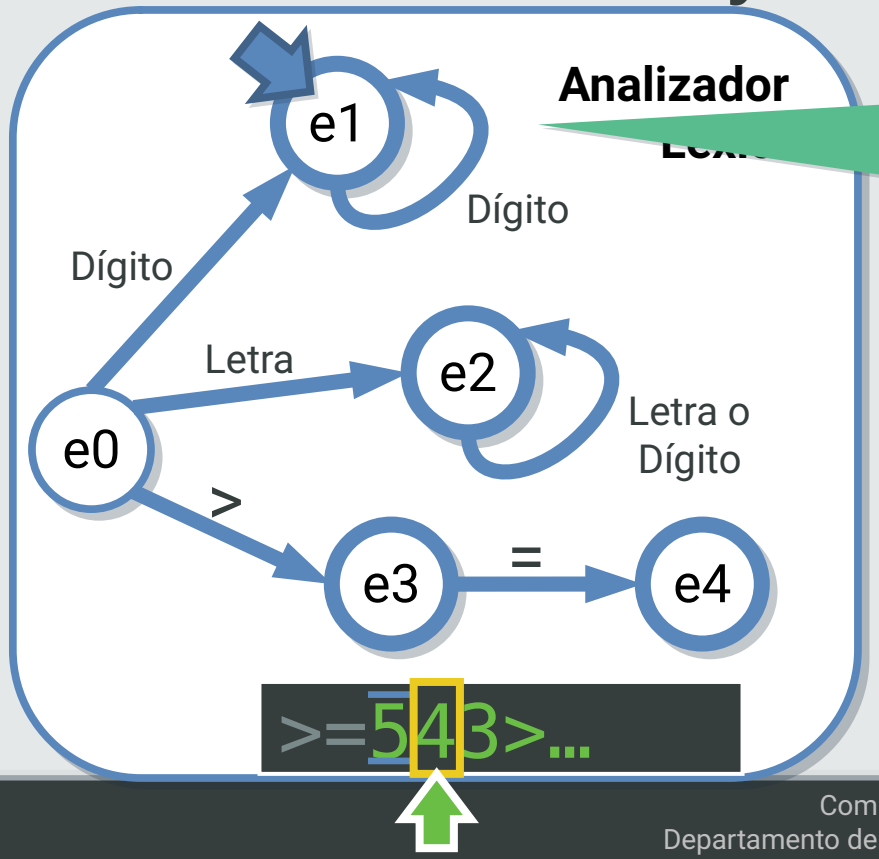
ente va a contener **mas de un token**, pero
onoce y retorna de a uno a la vez



Next Token!

Pedidos y Retornos de Tokens

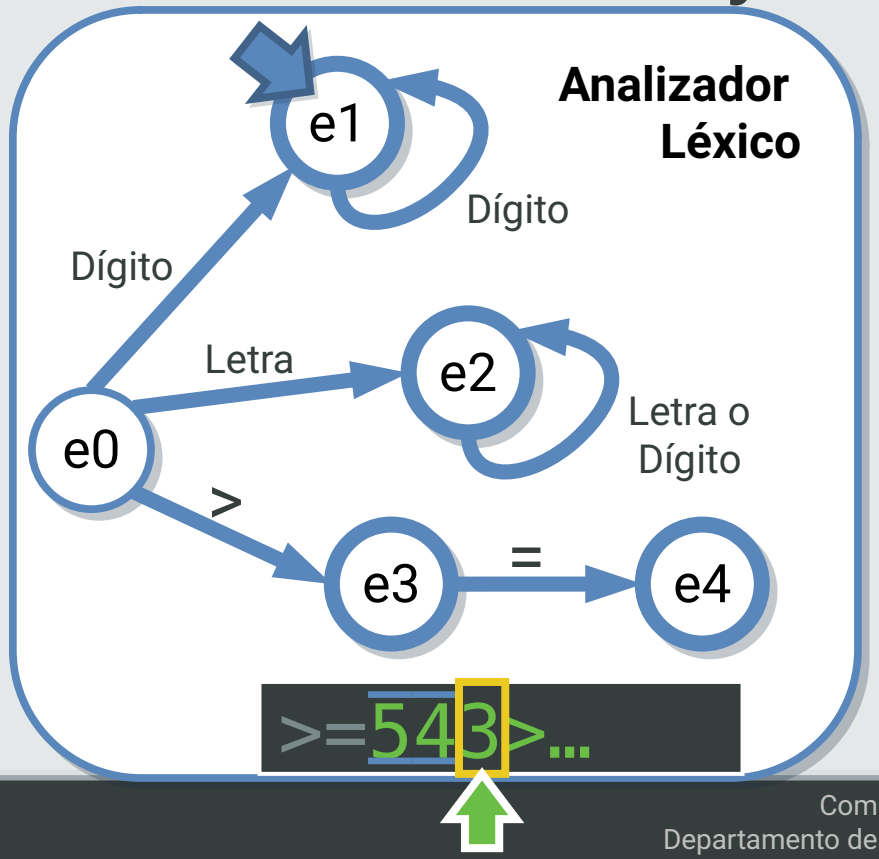
- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



Acá estamos en una situación similar a la anterior, si bien estamos en un estado terminador podemos seguir agregando caracteres al lexema

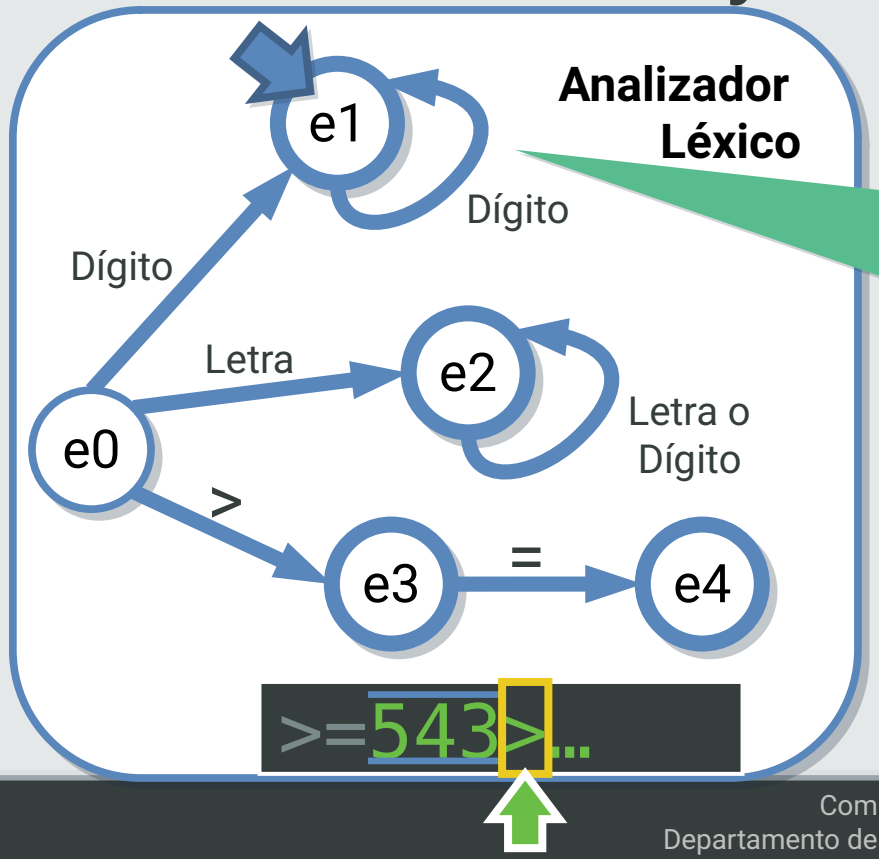
Pedidos y Retornos de Tokens

- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



Pedidos y Retornos de Tokens

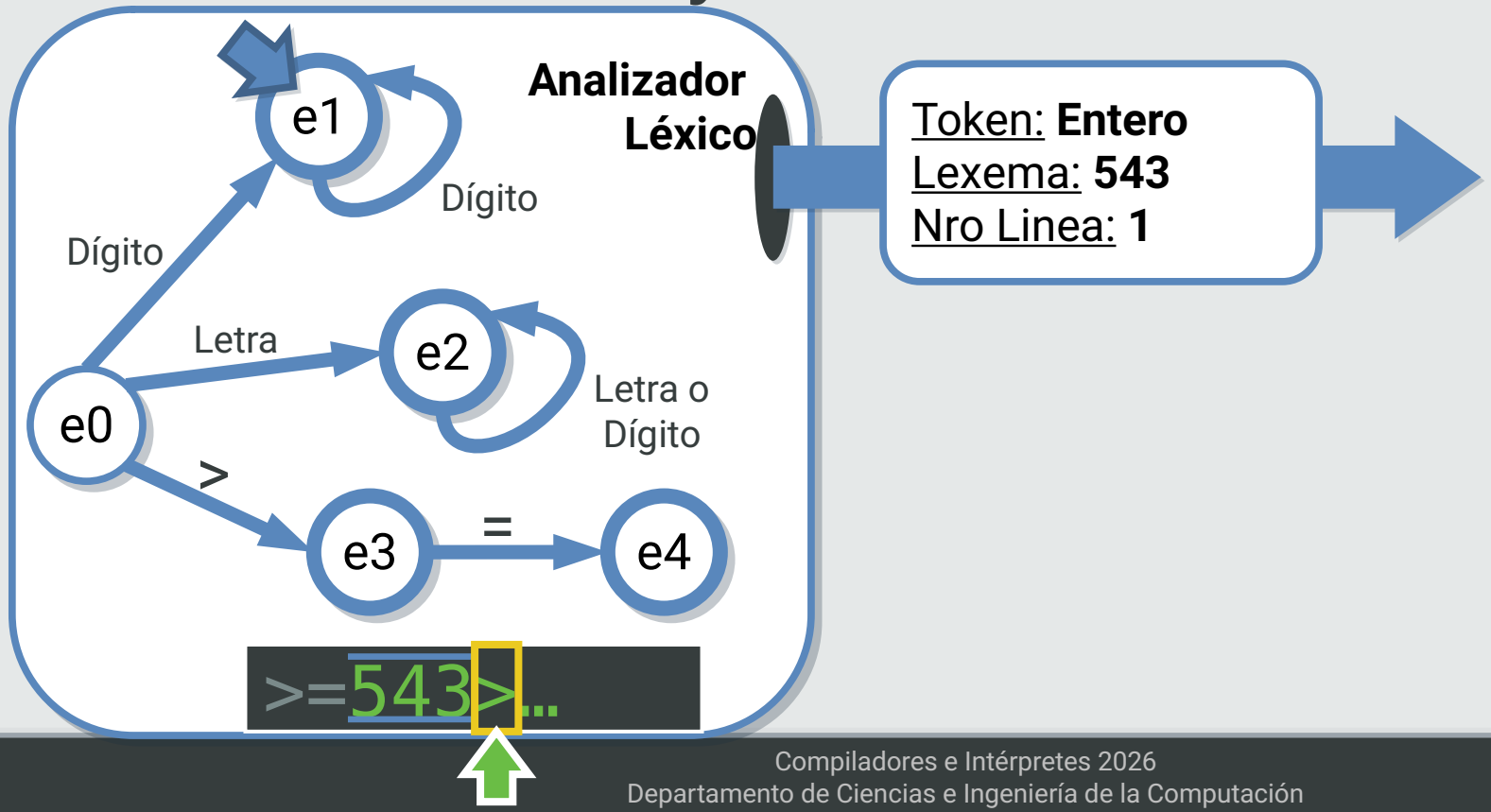
- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



Ahora llegue a una situación donde no puedo seguir agregando caracteres porque con “>” no hay transiciones desde e1… pero como e1 es un estado terminador retorno el token entero

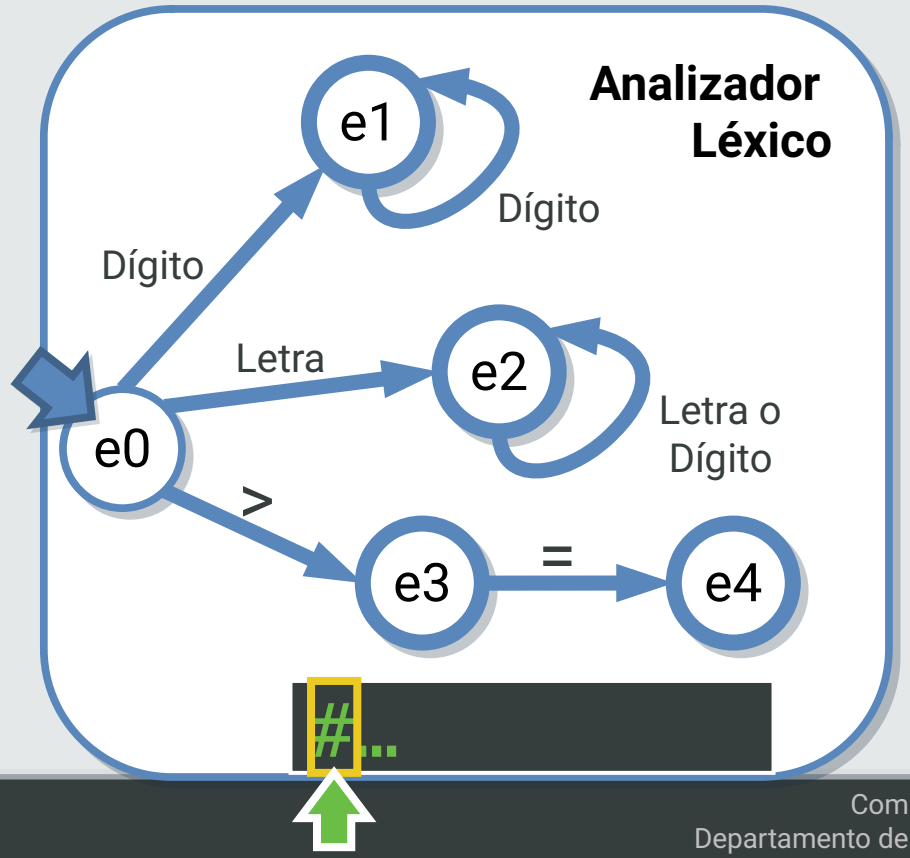
Pedidos y Retornos de Tokens

- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



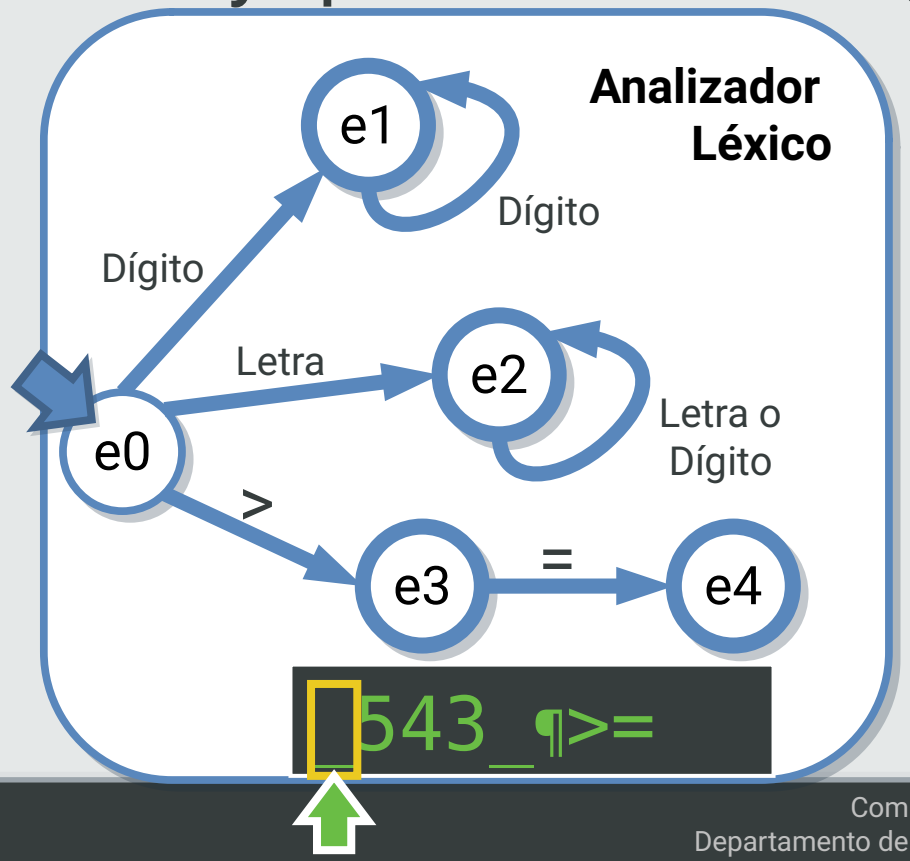
Errores Léxicos

- ¿Cómo nos damos cuenta que se produjo un Error Léxico?



Salteando Espacios en Blanco y Comentarios

- Como vimos los **blancos** y los **comentarios** no son tokens, pero hay que saltearlos… ¿Cómo Hacemos?

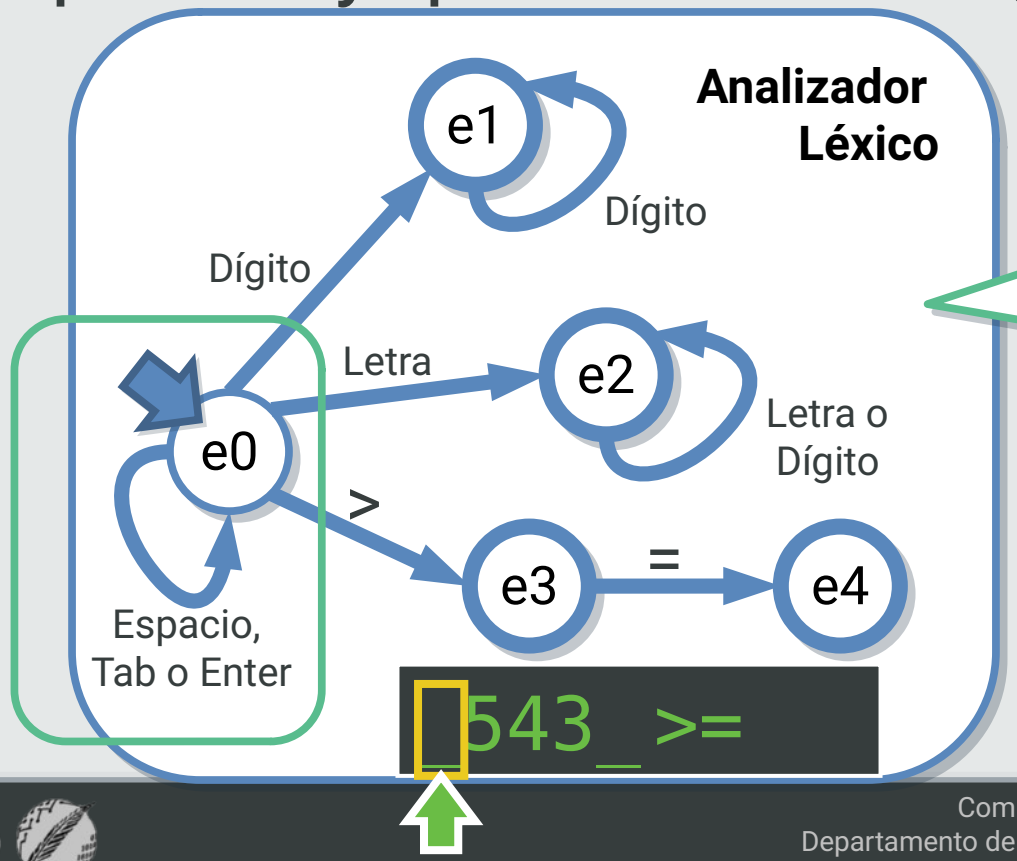


Concentrémonos en los blancos (espacios, tabs y enters)…

Son caracteres pueden usarse para separar tokens y hay que saltearlos, pero también los tengo que consumir...

Salteando Espacios en Blanco y Comentarios

- Como vimos los blanco y los comentarios no son tokens, pero hay que saltearlos… ¿Cómo Hacemos?

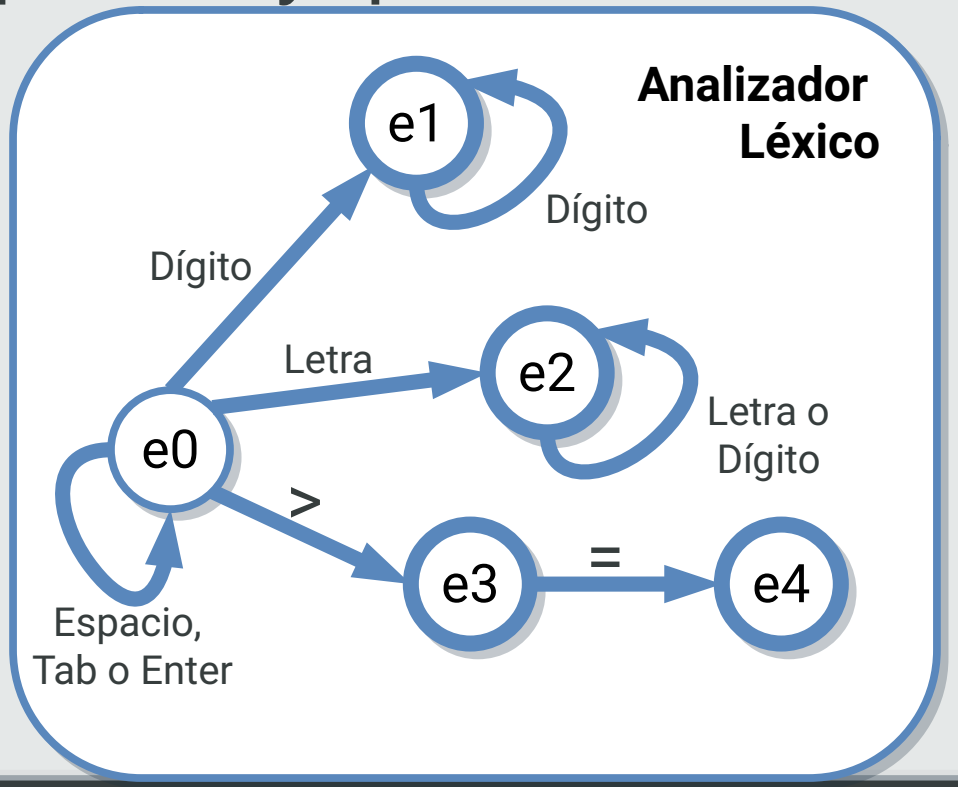


La idea es que mientras vengan blancos, los voy consumiendo y me mantengo en e0

Algo importante a tener en cuenta es que los blancos no forman parte los lexemas!

Salteando Espacios en Blanco y Comentarios

- Como vimos los blanco y los comentarios no son tokens, pero hay que saltearlos… ¿Cómo Hacemos?



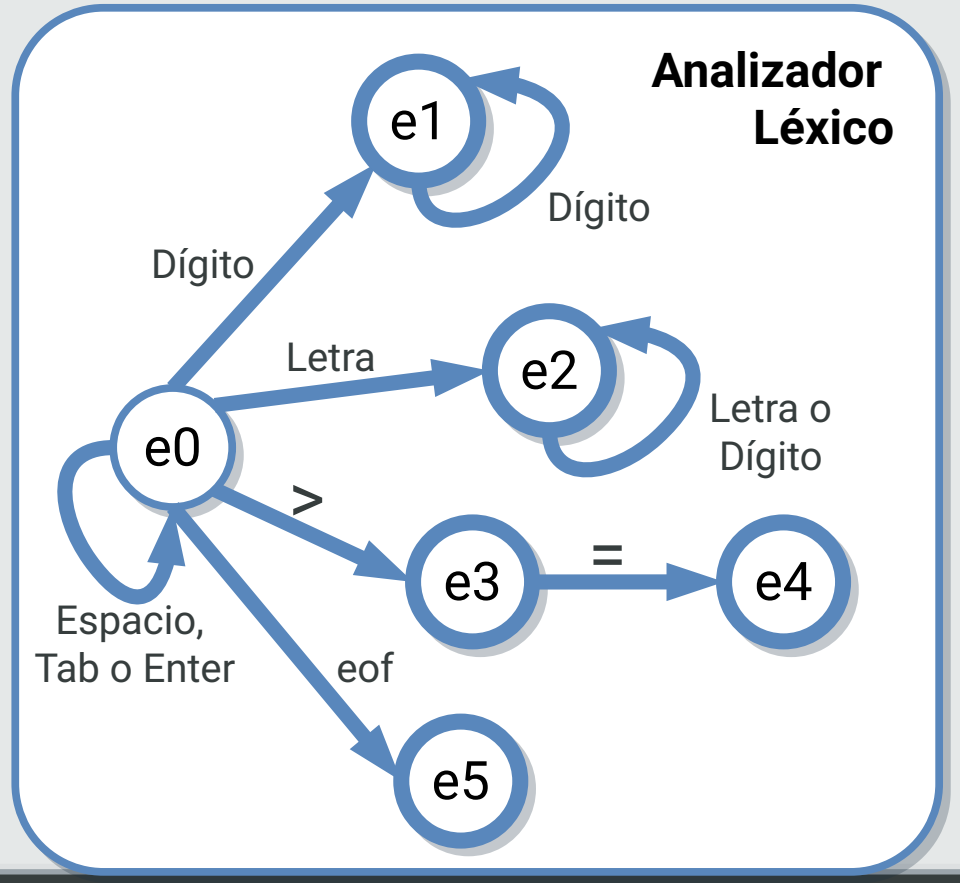
También hay que lidiar con los comentarios… Tenemos que saltearlos y controlar que los comentarios multi-linea estén cerrdados!

Esto queda para que lo piensen ustedes...



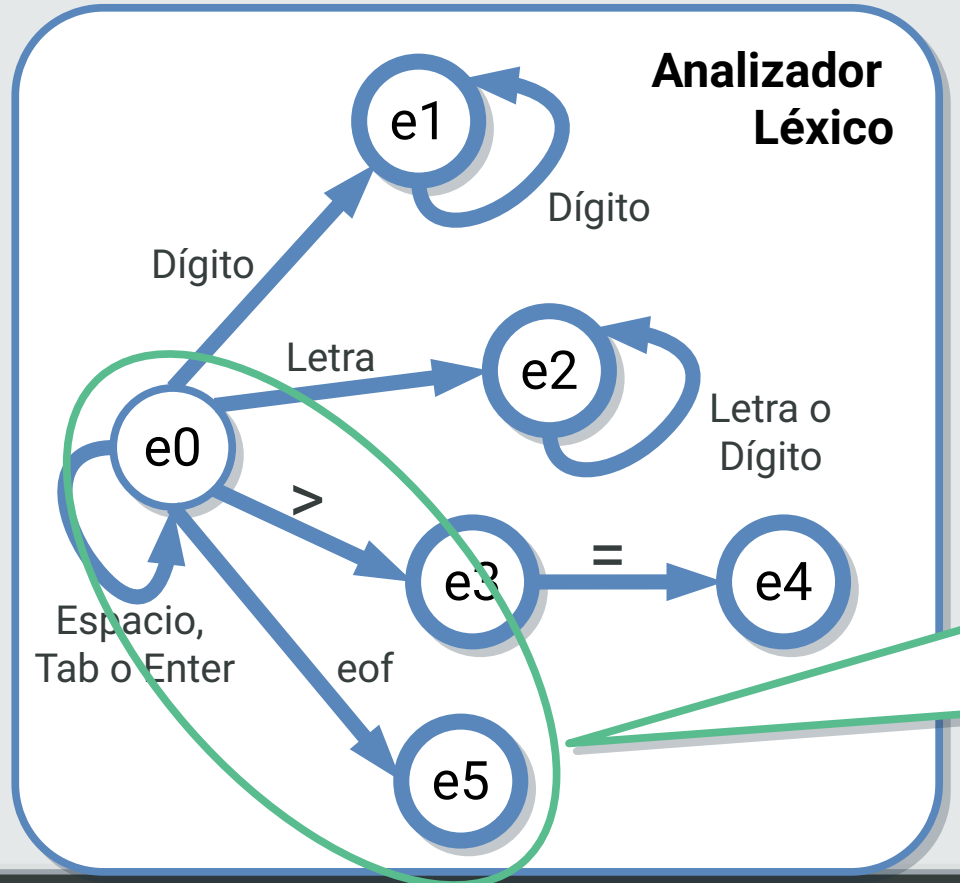
End of File

- El fin de Archivo es un Token mas



End of File

- El fin de Archivo es un Token mas



A diferencia de un token común cuando hacemos la transición por end of file (eof), no consumimos el carácter

Compiladores e Intérpretes
Analizador Léxico – Implementación

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

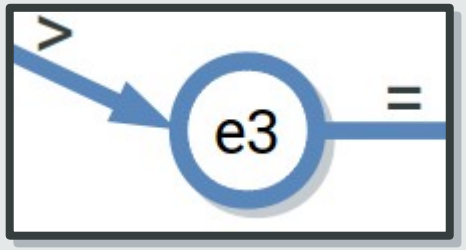
Implementando el Analizador Léxico

- A partir del AF que diseñamos recién hay **tres estrategias** para **implementar** el Analizador Léxico:
  - **Método x Estado**
  - Swtich/Case
  - Tabla

Nosotros vamos a concentrarnos en esta estrategia

Método por Estado – Implementación

- La **intuición** de esta estrategia es la siguiente
  - Por **cada estado** vamos a hacer **un método**





```
Token e3(){...}
```

  - El trabajo del AF se **simulará** con el **encadenamiento de llamadas** entre los metodos de los estados
  - Cada **método** va **simular** el **comportamiento AF** cuando se encuentra en el **estado** que representa
  -

Pedidos y Retornos de Tokens

- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



Pedidos y Retornos de Tokens

- El programa fuente va a contener **mas de un token**, pero nuestro **AF reconoce y retorna de a uno a la vez**



Método por Estado – Implementación

- La **invocación** de los estados **arranca** con el **pedido de un token**, donde llamaremos al estado **e0** y arrancando con un **lexema vació**

```
public Token proximoToken(){
    lexema = "";
    return e0()
}
```

Noten que e0() nos devuelve el token que se termino creando, y fueron devolviendo todos los estados por los que se paso

Esquema General de como Implementar el AF

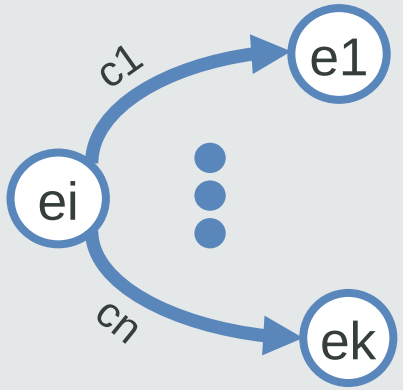
- Teniendo en cuenta todas esas consideraciones podemos pensar en **esquemas generales** para traducir los **estados no terminadores** y los **estados terminadores** a métodos
- A continuación vamos a ver como son estos esquemas

Es importante resaltar es que estos esquemas **no aplican exactamente igual** a los comportamientos excepcionales como el manejo **de blancos, comentarios y end of file**

Esquema General de como Implementar el AF

- Para cada estado *no terminador* **ei**, creamos una método **ei()**:

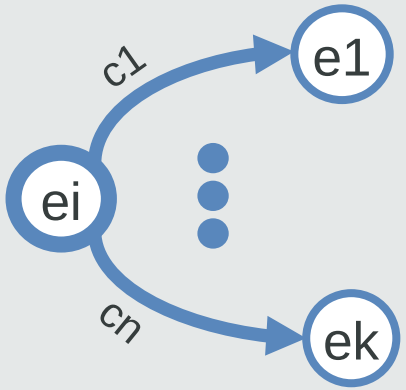
Si *caracterActual* = c1
    Agregar *caracterActual* a *lexemaActual*
    Actualizar *caracterActual*
    retornar el resultado de **e1()**
...
Sino Si *caracterActual* = cn
    Agregar *caracterActual* a *lexemaActual*
    Actualizar *caracterActual*
    retornar el resultado de **ek()**
Sino
    ERROR!



Esquema General de como Implementar el AF

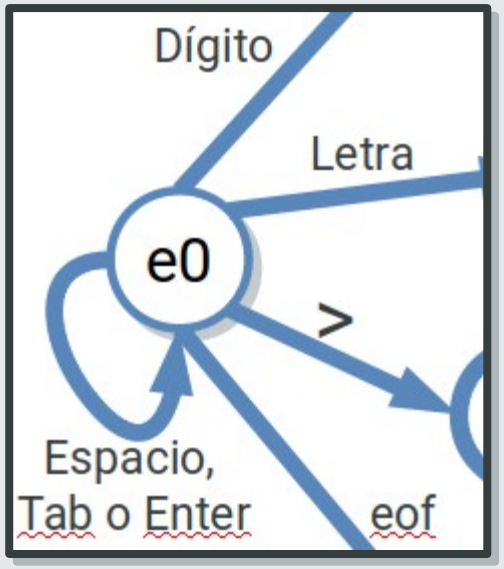
- Para cada estado *terminador* **ei**, creamos una método **ei()**:

Si *caracterActual* = c1
    Agregar *caracterActual* a *lexemaActual*
    Actualizar *caracterActual*
    retornar el resultado de **e1()**
...
Sino Si *caracterActual* = cn
    Agregar *caracterActual* a *lexemaActual*
    Actualizar *caracterActual*
    retornar el resultado de **em()**
Sino
    retornar *token* asociado a **ej**



Método por Estado – Implementación

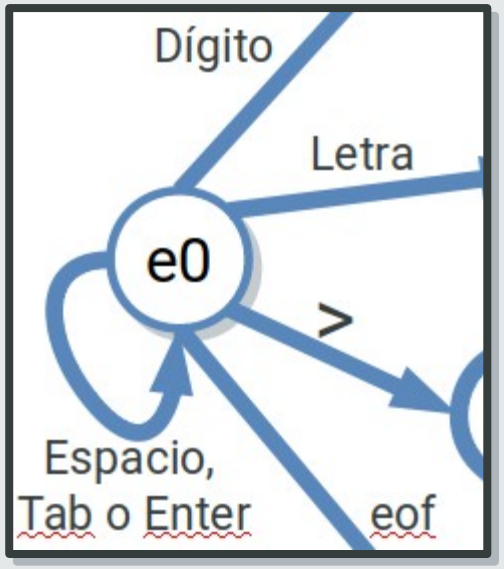
- Siguiendo lo que vimos en antes si estamos en un **estado no terminador** y si, en base al carácter actual, **no podemos aplicar** una transición debemos reportar un **error**




```
Token e0(){
    if(Character.isDigit(caracterActual)){
        ...
    }
    else{
        throw new ExcepcionLexica(lexema, nroLinea);
    }
}
```

Método por Estado – Implementación

- Siguiendo lo que vimos en antes si estamos en un **estado no terminador** y si, en base al **podemos aplicar** una transicio **error**

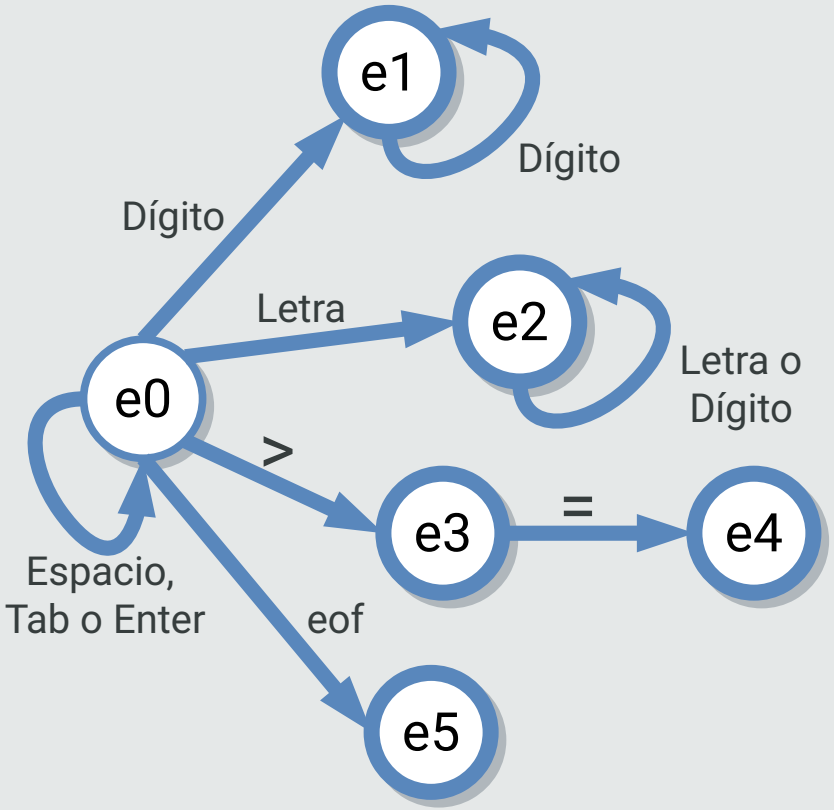



```
Token e0(
    if(Character.
        ...
    }
    else{
        throw new ExcepcionLexica(lexema, nroLinea);
    }
}
```

En general, al detectar el error el léxico lanza una excepción que es capturada por el modulo principal (quien contiene al léxico) para devolver/reportar el mensaje de error adecuada.

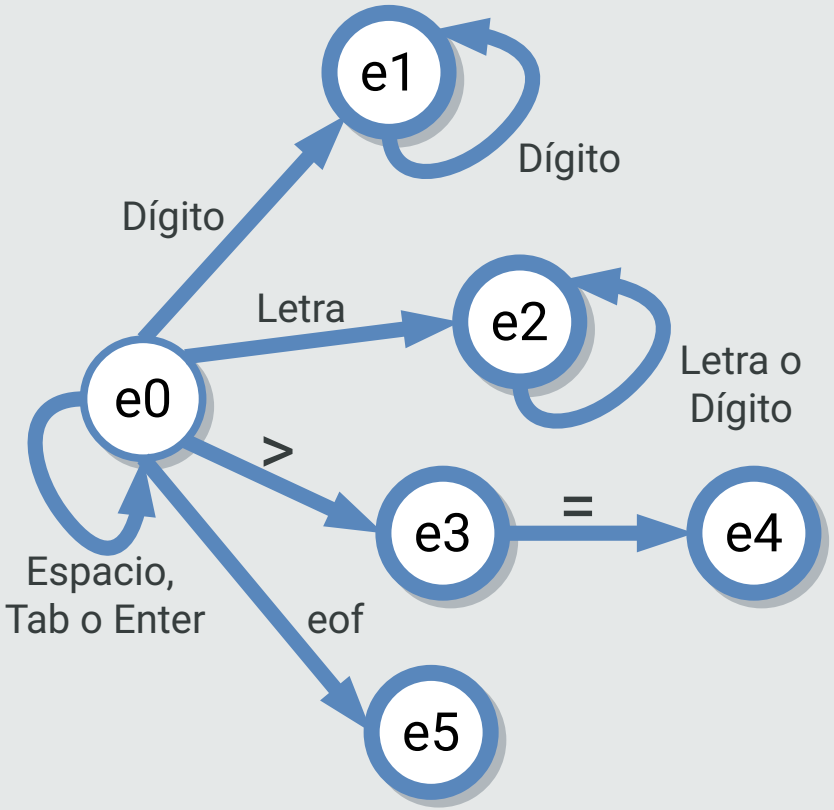
Ejemplo Completo

- Veamos la implementación en base al AF hecho en 2.3



Ejemplo Completo

- Veamos la implementación en base al AF hecho en 2.3



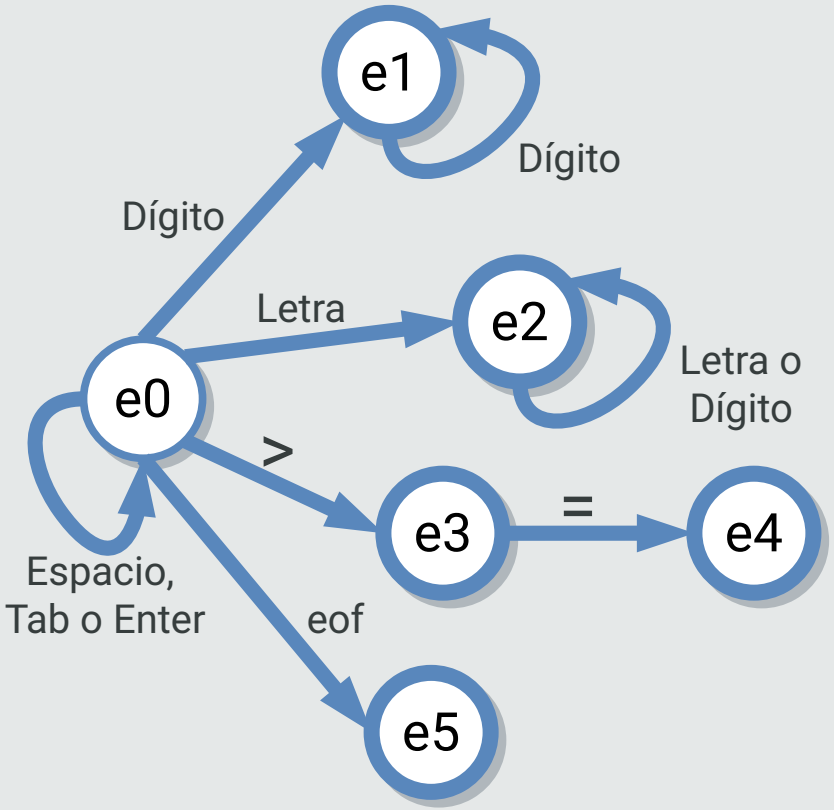

```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

    ...
}
```

Primero, en un lenguaje OO el Analizador Léxico probablemente va ser un objeto

Ejemplo Completo

- Veamos la implementación en base al AF hecho en 2.3




```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

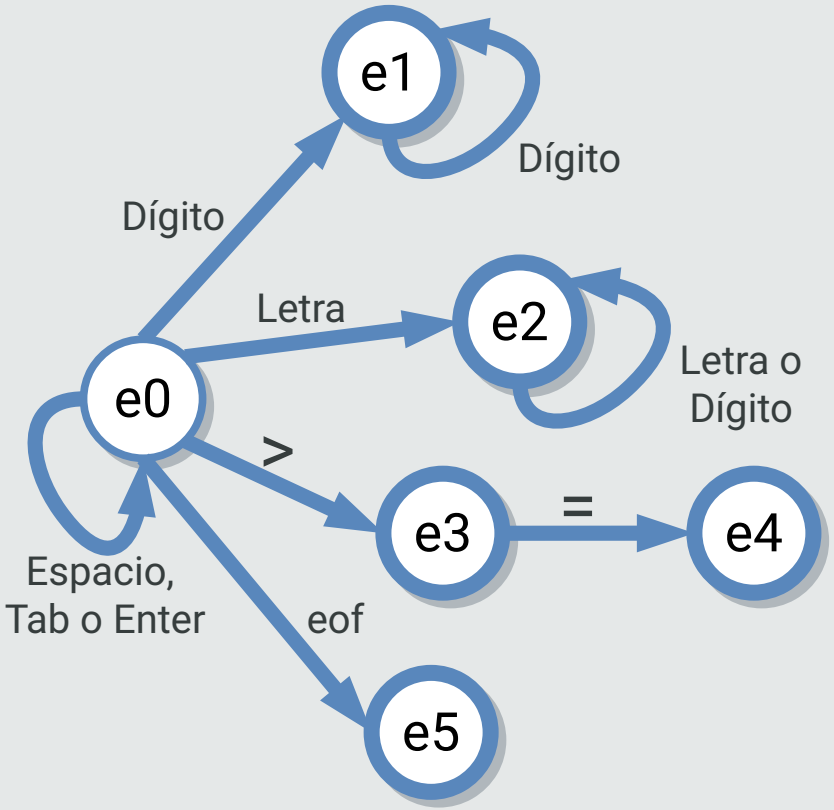
    ...
}
```

Primero, en un lenguaje OO el Analizador Léxico probablemente va ser un objeto

Vamos a usar atributos para llevar cuenta del carácter que actualmente estamos viendo, el lexema que venimos armando y el gestor de archivos

Ejemplo Completo

- Veamos la implementación en base al AF hecho en 2.3



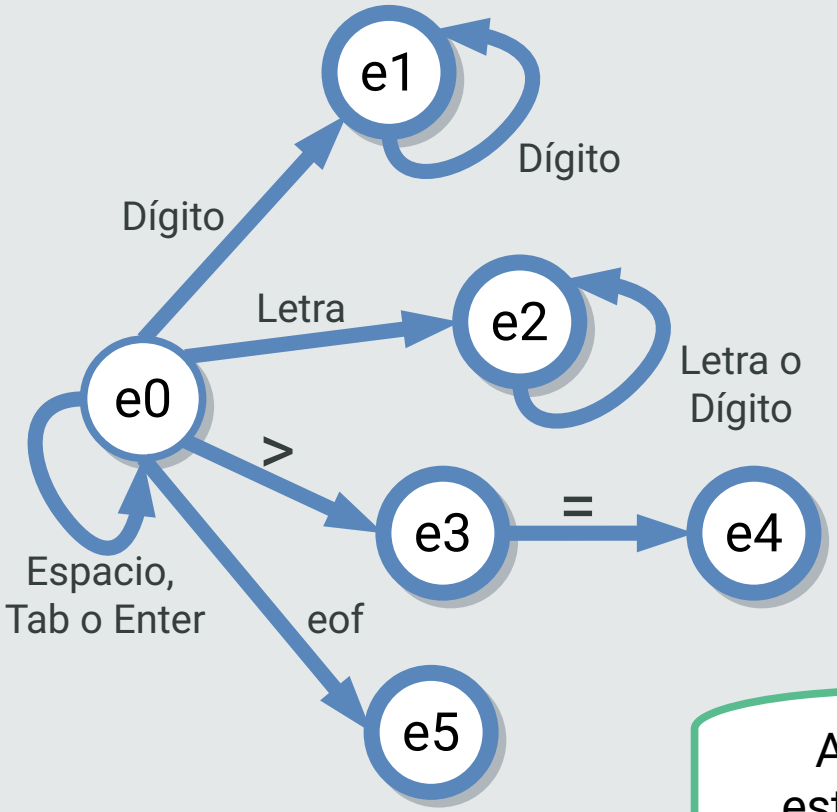

```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

    public Token proximoToken(){
        lexema = "";
        return e0()
    }
```

La interfaz del Léxico con otros componentes va ser mediante el servicio proximoToken() que vimos antes

Ejemplo Completo

- Veamos la implementación en base al AF hecho en 2.3




```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

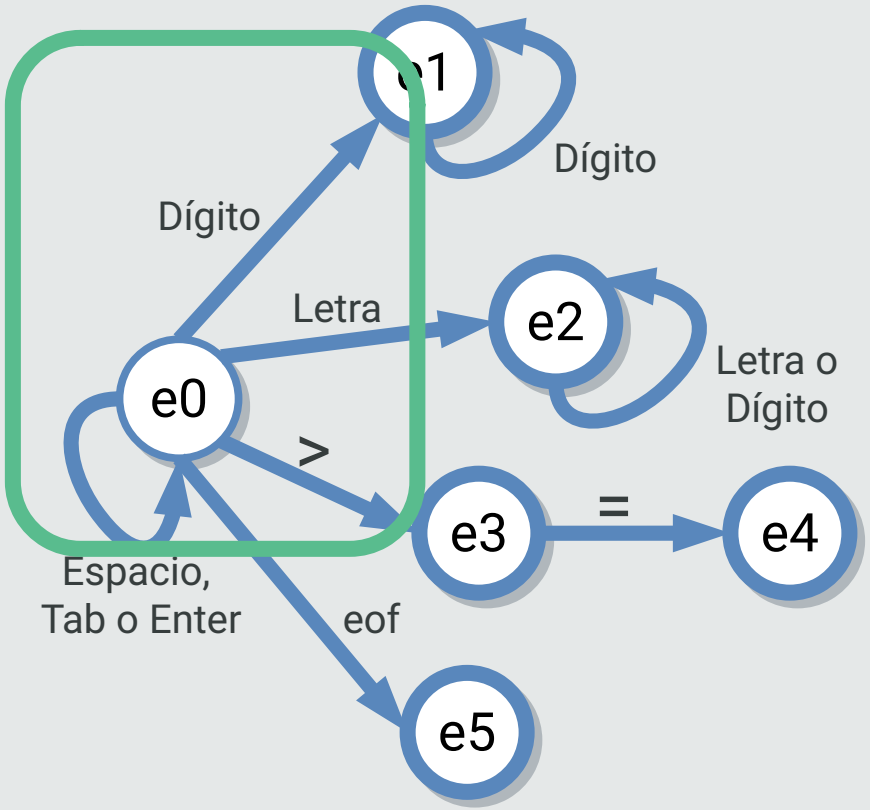
    public Token proximoToken(){...}

    private void actualizarLexema(){
        lexema = lexema + caracterActual;
    }
    private Cacho actualizarCaracterActual(){
        caracterActual = gestorDeFuente.proximoCaracter();
    }
```

Antes de arrancar con los estados vamos a usar estos dos métodos auxiliares para que el código de los métodos de los estados nos quede mas limpio

Ejemplo Completo

- Veamos la implementa



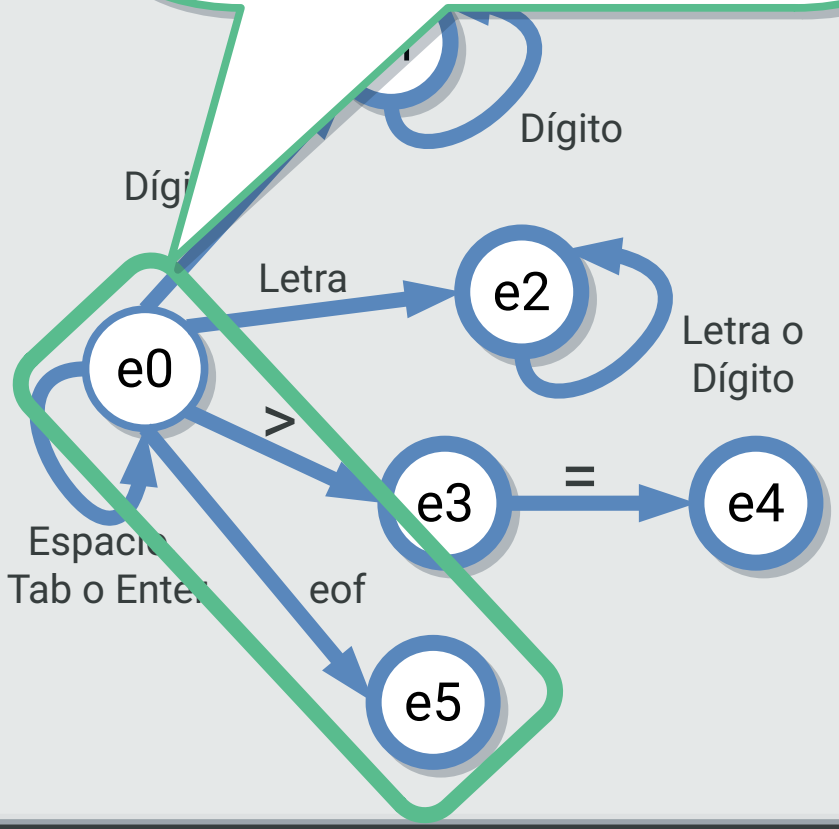

```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeF

    public Token proximoTo      (){...}
    private void actuali        Lexema(){...}
    private void actua          arCaracterActual(){...}

    private Token e0(){
        if(Character.isDigit(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e1();
        }else if(Character.isLetter(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e2();
        }else if(caracterActual == '>'){
            actualizarLexema();
            actualizarCaracterActual();
            return e3();
        }...
```

Arranquemos con la implementación de e0… Para las primeras tres transiciones seguimos el esquema general

Como mencionamos el manejo del **end of file** tiene un trato especial donde **no hay lexema y no hay que leer un nuevo carácter del fuente**




```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

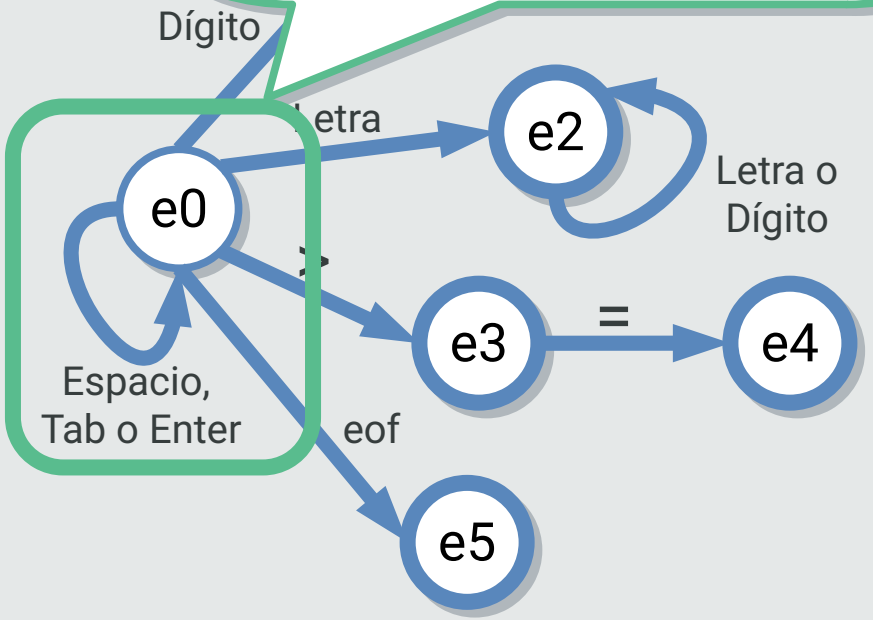
    public Token proximoToken(){...}
    private void actualizarLexema(){...}
    private void actualizarCaracterActual(){...}

    private Token e0(){
        if(Character.isDigit(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e1();
        }else if(Character.isLetter(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e2();
        }else if(caracterActual == '>'){
            actualizarLexema();
            actualizarCaracterActual();
            return e3();
        }else if(gestorDeFuente.esEOF(caracterActual)){
            return e5();
        }...
```

Ejemp

- ac

También como mencionamos los **blancos** hay que pasarlos ignorandolos. Es decir hay que avanzar la lectura pero no agregarlos al lexema actual



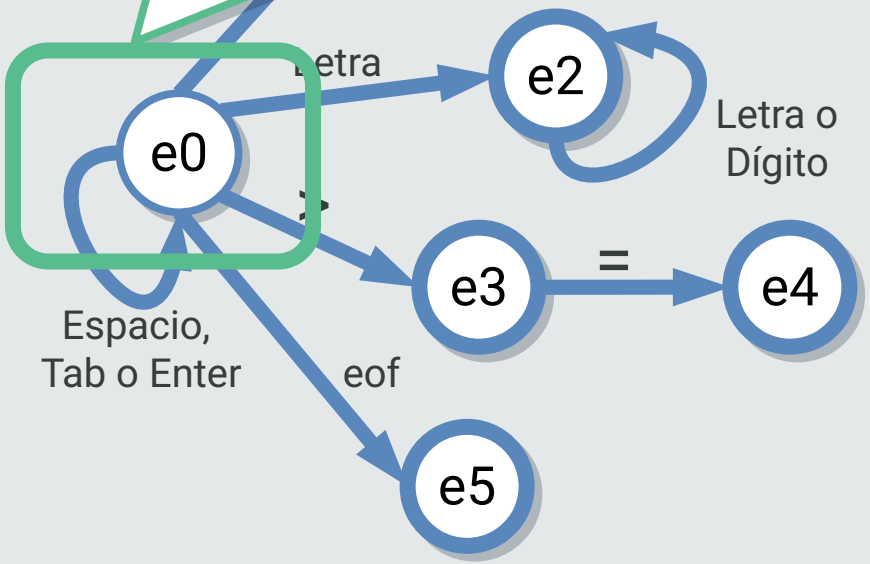

```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

    public Token proximoToken(){...}
    private void actualizarLexema(){...}
    private void actualizarCaracterActual(){...}

    private Token e0(){
        if(Character.isDigit(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e1();
        }else if(Character.isLetter(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e2();
        }else if(caracterActual == '>'){
            actualizarLexema();
            actualizarCaracterActual();
            return e3();
        }else if(gestorDeFuente.esEOF(caracterActual)){
            return e5();
        }else if(Character.isWhitespace()){
            actualizarCaracterActual();
            return e0();
        }...
```

Eje

Por último para e0, como es un estado no terminador si con el carácter actual no podemos transicionar, detectamos y propagamos un error léxico



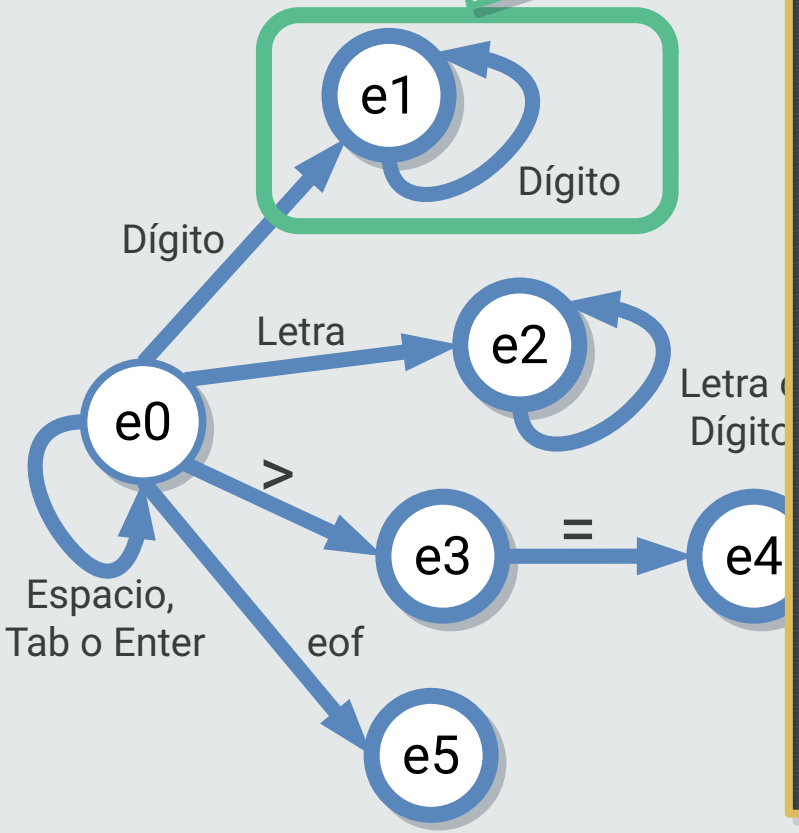

```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

    public Token proximoToken(){...}
    private void actualizarLexema(){...}
    private void actualizarCaracterActual(){...}

    private Token e0(){
        if(Character.isDigit(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e1();
        }else if(Character.isLetter(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e2();
        }else if(caracterActual == '>'){
            actualizarLexema();
            actualizarCaracterActual();
            return e3();
        }else if(gestorDeFuente.esEOF(caracterActual)){
            return e5();
        }else if(Character.isWhitespace()){
            actualizarCaracterActual();
            return e0();
        }else{
            actualizarLexema();
            throw new ExcepcionLexica(lexema,gestorDeFuente.nroLinea());
        }
    }
```

emplo Completo

e1 sigue el esquema general de un estado terminador. Mientras vengan dígitos sigo en e1, si viene otra cosa retrono el entero que arme




```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

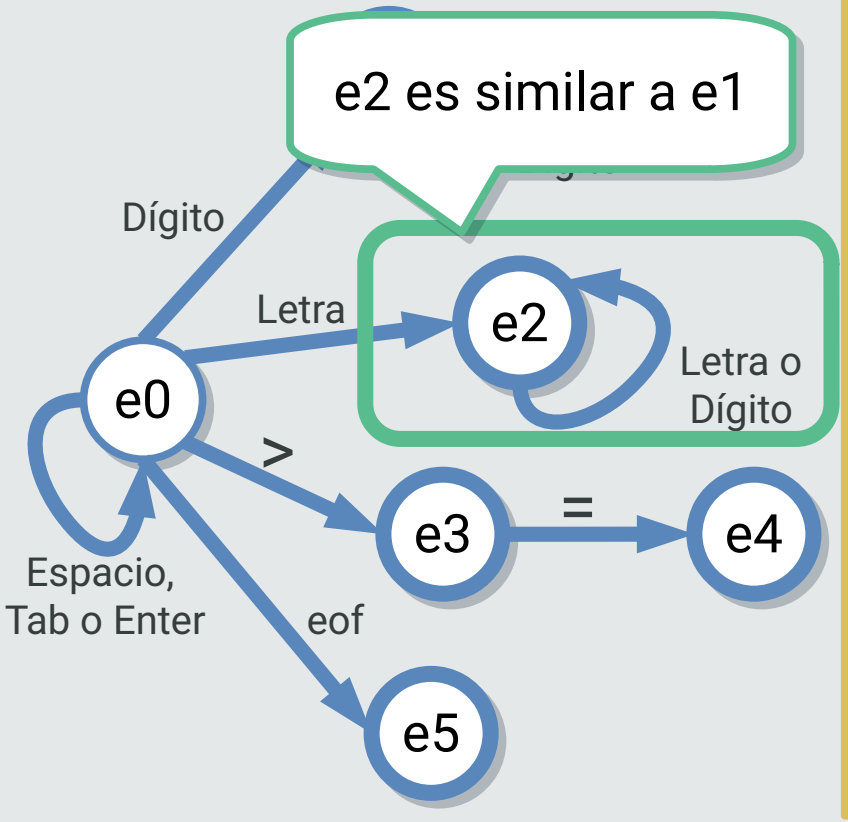
    public Token proximoToken(){...}
    private void actualizarLexema(){...}
    private void actualizarCaracterActual(){...}

    private Token e0(){...}

    private Token e1(){
        if(Character.isDigit(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e1();
        }else{
            return new Token("entero", lexema, gestorDeFuente.nroLinea());
        }
    }
```

Ejemplo Completo

- Veamos la implementación en base al AF hecho en 2.3




```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

    public Token proximoToken(){...}
    private void actualizarLexema(){...}
    private void actualizarCaracterActual(){...}

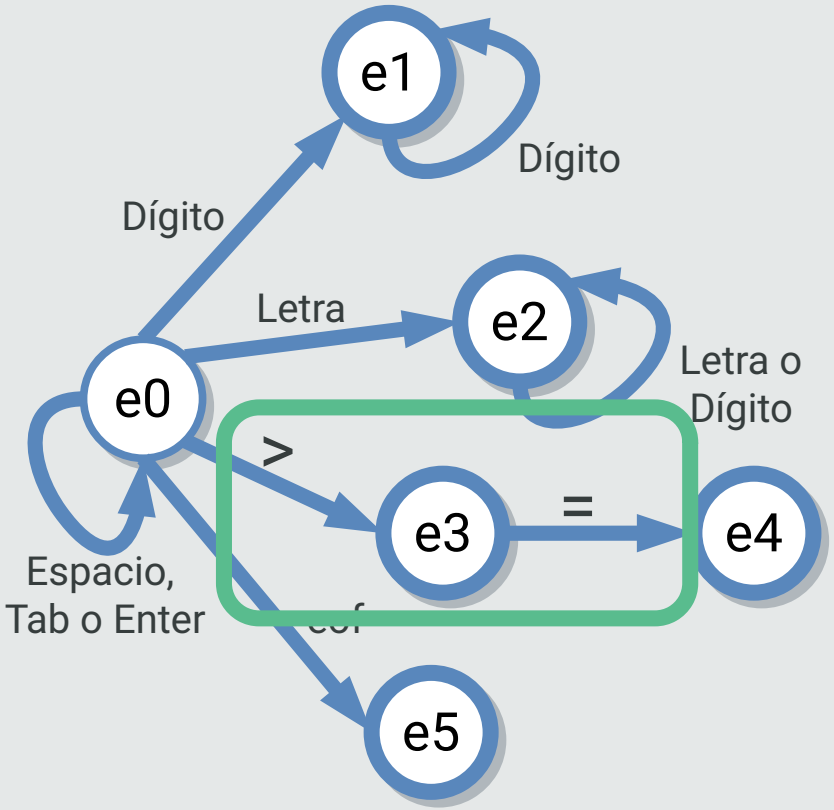
    private Token e0(){...}

    private Token e1(){...}

    private Token e2(){
        if(Character.isLetter(caracterActual) || Character.isLetter(caracterActual)){
            actualizarLexema();
            actualizarCaracterActual();
            return e2();
        }else{
            return new Token("identificador", lexema, gestorDeFuente.nroLinea());
        }
    }
```

Ejemplo Completo

- Veamos la impleme




```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

    public Token proximoToken(){...}
    private void actualizarLexema(){...}
    private void actualizarCaracterActual(){...}

    private Token e0(){...}

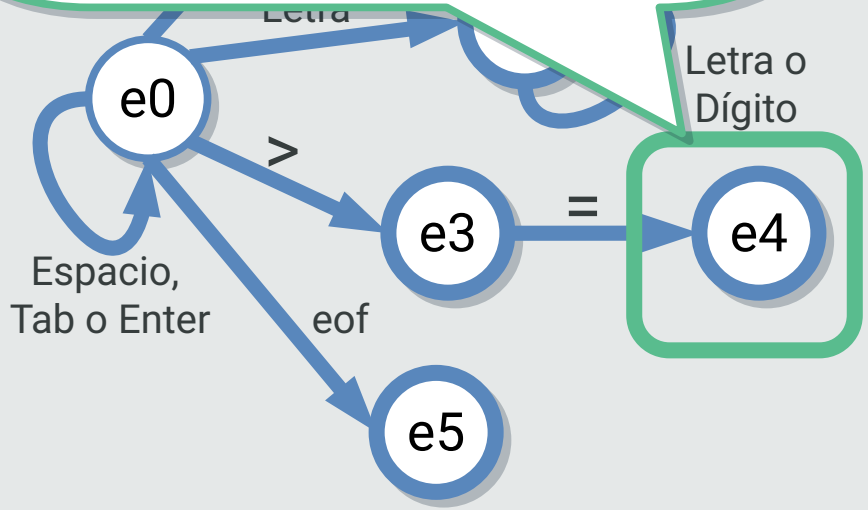
    private Token e1(){...}

    private Token e2(){...}

    private Token e3(){
        if(caracterActual == '='){
            actualizarLexema();
            actualizarCaracterActual();
            return e4();
        }else{
            return new Token("Mayor", lexema, gestorDeFuente.nroLinea());
        }
    }
```

Ejemplo Completo

e4 sigue el esquema general de un estado terminador, pero como no hay ninguna transición posible sea cual fuera el carácter actual tengo que retornar el token




```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

    public Token proximoToken(){...}
    private void actualizarLexema(){...}
    private void actualizarCaracterActual(){...}

    private Token e0(){...}

    private Token e1(){...}

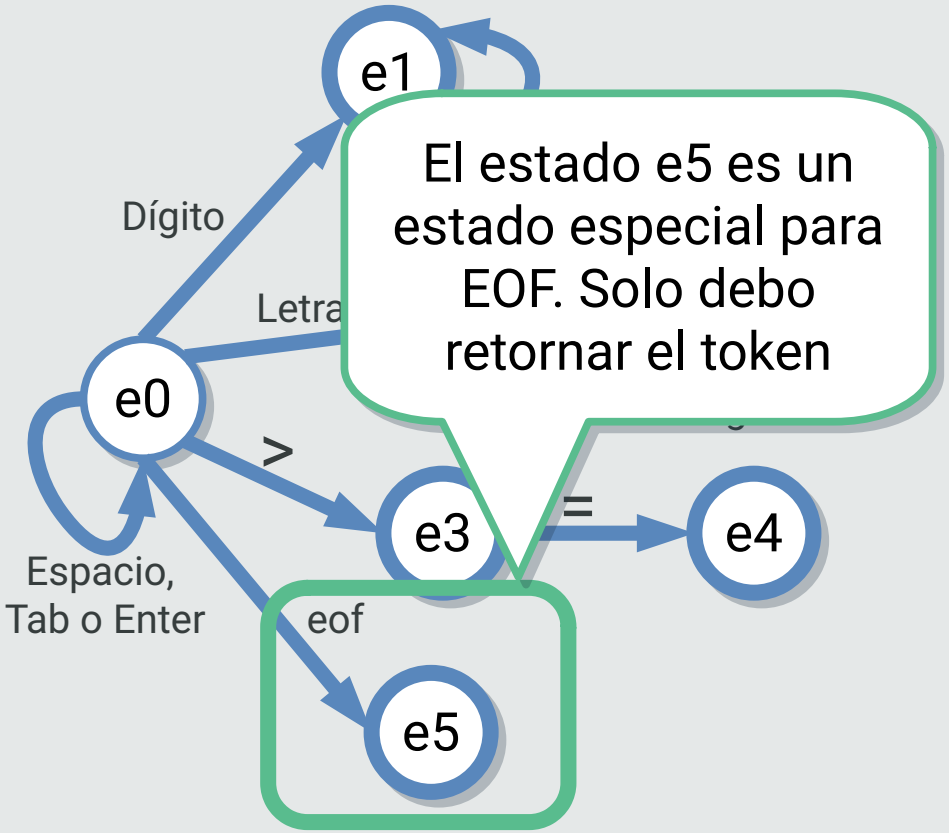
    private Token e2(){...}

    private Token e3(){...}

    private Token e4(){
        return new Token("MayorIgual", lexema, gestorDeFuente.nroLinea());
    }
```

Ejemplo Completo

- Veamos la implementa




```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

    public Token proximoToken(){...}
    private void actualizarLexema(){...}
    private void actualizarCaracterActual(){...}

    private Token e0(){...}

    private Token e1(){...}

    private Token e2(){...}

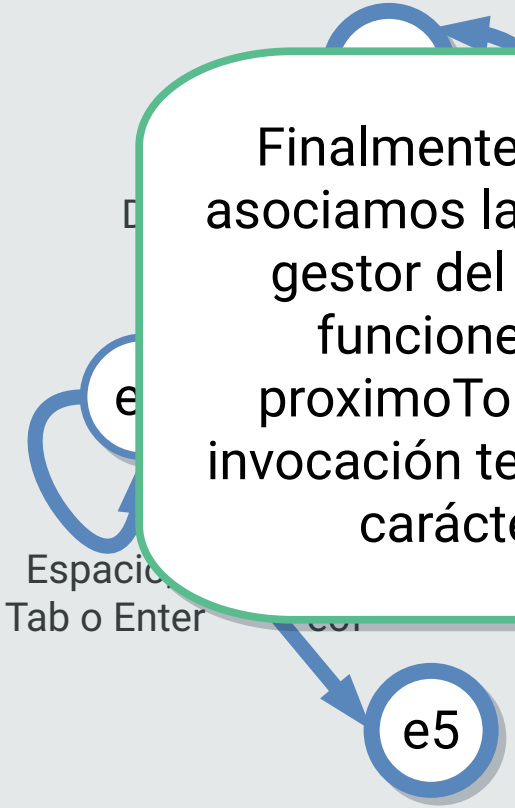
    private Token e3(){...}

    private Token e4(){...}

    private Token e5(){
        return new Token("EOF", lexema, gestorDeFuente.nroLinea());
    }
```

Ejemplo Completo

- Veamos la implementación



Finalmente, en el constructor asociamos la dependencia con el gestor del fuente y para que funcione correctamente proximoToken() en su primer invocación tenemos que tener un carácter actual leído

```
class AnalizadorLexico{
    String lexema;
    char caracterActual;
    GestorDeArchivo gestorDeFuente

    public Token proximoToken(){...}
    private void actualizarLexema(){...}
    private void actualizarCaracterActual(){...}

    private Token e0(){...}

    private Token e1(){...}

    private Token e2(){...}

    private Token e3(){...}

    private Token e4(){...}

    private Token e5(){...}

    public AnalizadorLexico(GestorDeArchivo gestor){
        gestorDeFuente = gestor;
        actualizarCaracterActual();
    }
}
```

Ejemplo Completo



Mmmm… ¿Puede ser que invocar a proximoToken lleve a tener muchas llamadas a métodos? … por ejemplo si tengo 20 espacios y pegado un numero de 10 dígitos proximoToken() va a terminar llevándonos a 30 llamadas **¿No se puede hacer mas eficiente?**

Si, son muchas llamadas… lo de la **eficiencia es discutible** (depende del compilador del lenguaje que estemos usando para implementar).

```
ass AnalizadorLexico{
  String lexema;
  char caracterActual;
  GestorDeArchivo gestorDeFuente

  public Token proximoToken(){...}
  private void actualizarLexema(){...}
  private void actualizarCaracterActual(){...}

  private Token e0(){...}

  private Token e1(){...}

  private Token e2(){...}

  private Token e3(){...}

  private Token e4(){...}

  private Token e5(){...}

  public AnalizadorLexico(GestorDeArchivo gestor){
      gestorDeFuente = gestor;
      actualizarCaracterActual();
  }
}
```

Ejemplo Completo



Mmmm… ¿Puede ser que invocar a proximoToken lleve a tener muchas llamadas a métodos? … por ejemplo si tengo 20 espacios y pegado un numero de 10 dígitos proximoToken() va a terminar llevándonos a 30 llamadas **¿No se puede hacer mas eficiente?**

Si, son muchas llamadas… lo de la **eficiencia es discutible** (depende del compilador del lenguaje que estemos usando para implementar).

Existen otras estrategias para implementar el AF que nos achican el numero de llamadas, a costa de tener un código **mucho mas difícil de leer**… Veamos como es el esquema general de la basada en un gran **case/switch de estados**

```
ass AnalizadorLexico{
  String lexema;
  char caracterActual;
  GestorDeArchivo gestorDeFuente

  public Token proximoToken(){...}
  private void actualizarLexema(){...}
  private void actualizarCaracterActual(){...}

  private Token e0(){...}

  private Token e1(){...}

  private Token e2(){...}

  private Token e3(){...}

  e Token e4(){...}

  e Token e5(){...}

   AnalizadorLexico(GestorDeArchivo gestor){
  storDeFuente = gestor;
  tualizarCaracterActual();
  }
}
```

Antes de terminar: Palabras Clave

- En general **implementar las palabras claves** del lenguaje utilizando sus AF es **problemático** porque:
  - Requiere **muchos estados** (casi uno por letra de la palabra clave)
  - Hay que **fusionar** esos estados entre sí y con los de **identificadores**
- En la mayoría de los lenguajes las **palabras clave son** prácticamente **identificadores** “especiales”…



¿Cómo podemos aprovechar esto?

Antes de terminar: Palabras Clave

- En general **implementar las palabras claves** del lenguaje utilizando sus AF es **problemático** porque:
  - Requiere **muchos estados** (casi uno por letra de la palabra clave)
  - Hay que **fusionar** esos estados entre sí y con los de **identificadores**
- En la mayoría de los lenguajes las **palabras clave son** prácticamente **identificadores** “especiales”…



¿Cómo podemos aprovechar esto?

La idea es tener una tabla con todos los lexemas de las palabras clave y cuando armamos un token identificador controlamos si es o no una palabra clave en la tabla y devolvemos el token que corresponda

Por ejemplo, en MiniJava “else” debería estar en la tabla, entonces cuando terminamos de armar un identificador con lexema “else” antes de retornarlo chequeamos en esa tabla y en vez de retornar un token identificador **retornamos un token pr_else**